



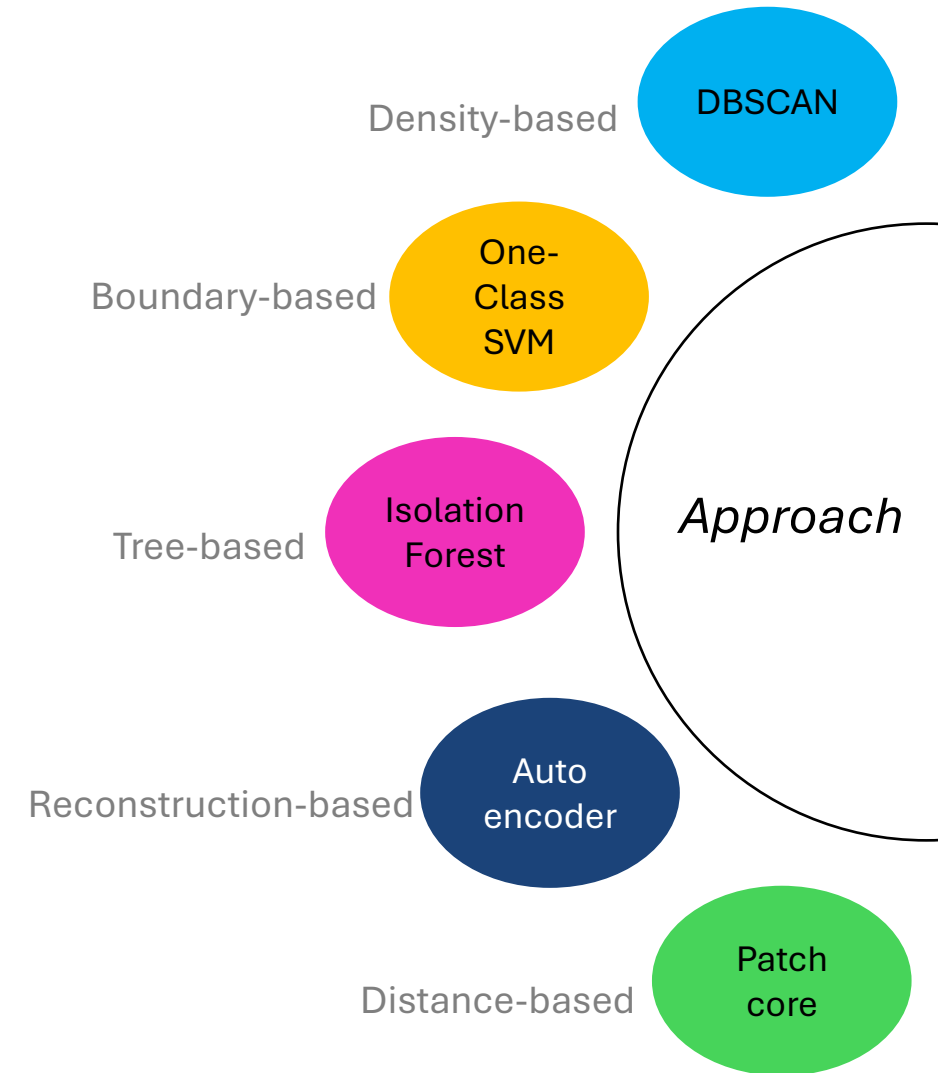
Frankfurt School

Comparative Study of Image Anomaly Detection Techniques

Minhung YANG, Liting HUANG, Xuan Tuan NGUYEN, Seonyoung KWON, Victoria Angela VIZMONTE
MADS 2024 Machine Learning 2 | May 8th, 2025

Research Objective

1. Investigate effective methods to solve the business problem of image anomaly detection in industrial products.
2. Understand the theoretical and practical characteristics of each method by comparing multiple ML/DL-based anomaly detection approaches.
3. Analyze how different techniques interpret 'normality' and respond to deviations when trained only on normal image data..



A Key to Reliable Automation

Anomaly Detection is essential in domains where:

- Abnormal cases are rare, diverse, or costly to label.
- Automation and early detection can save cost, time, and even lives.
- Defects are rare but critical in manufacturing, healthcare, security and finance.



Industrial Quality Control

- Detecting surface defects (scratches, dents, contamination) on metal, plastic, or electronic components.
- Avoids defective product shipment and reduces manual inspection cost.



Medical Imaging

- Identifying tumors or lesions in MRI/CT scans where abnormal cases are limited.
- Helps doctors detect rare but critical conditions with high sensitivity.



Security & Surveillance

- Detecting unusual behavior or objects in video feeds.
- Crucial in preventing safety incidents or intrusions.

Dataset & Common Methodology

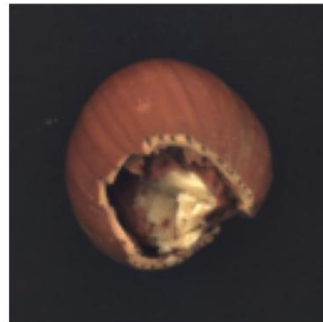
- All approaches are **unsupervised**, training exclusively on normal data.
- Feature extraction and anomaly scoring assume **no access to labeled defects** during training.
- **F1 Score** balances the risks of missing rare anomalies (low recall) and generating too many false alarms (low precision).

Object - Hazelnut

Normal



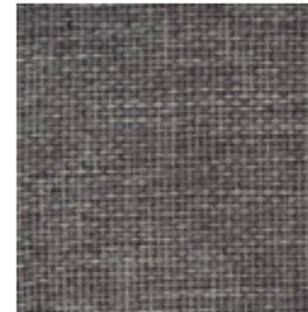
Anomaly



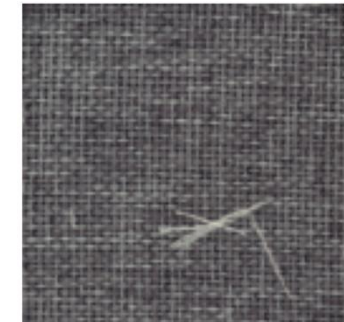
- **Train set:** 391 normal images (only “good” samples)
- **Test set:** 110 images (4 defect types)
- Coherent object images with uniform shape and color—relatively easy to learn normal patterns.

Material - Carpet

Normal



Anomaly



- **Train set:** 280 normal patches
- **Test set:** 117 patches (5 defect types)
- Highly variable texture and subtle defects—expected to challenge detection models and yield lower performance.

AUTO ENCODER



What is an Autoencoder?



Methodology : Neural Network

Operates without labeled data using unsupervised learning techniques.



Target : Data Compression

Creates efficient representations of input data in lower dimensions.



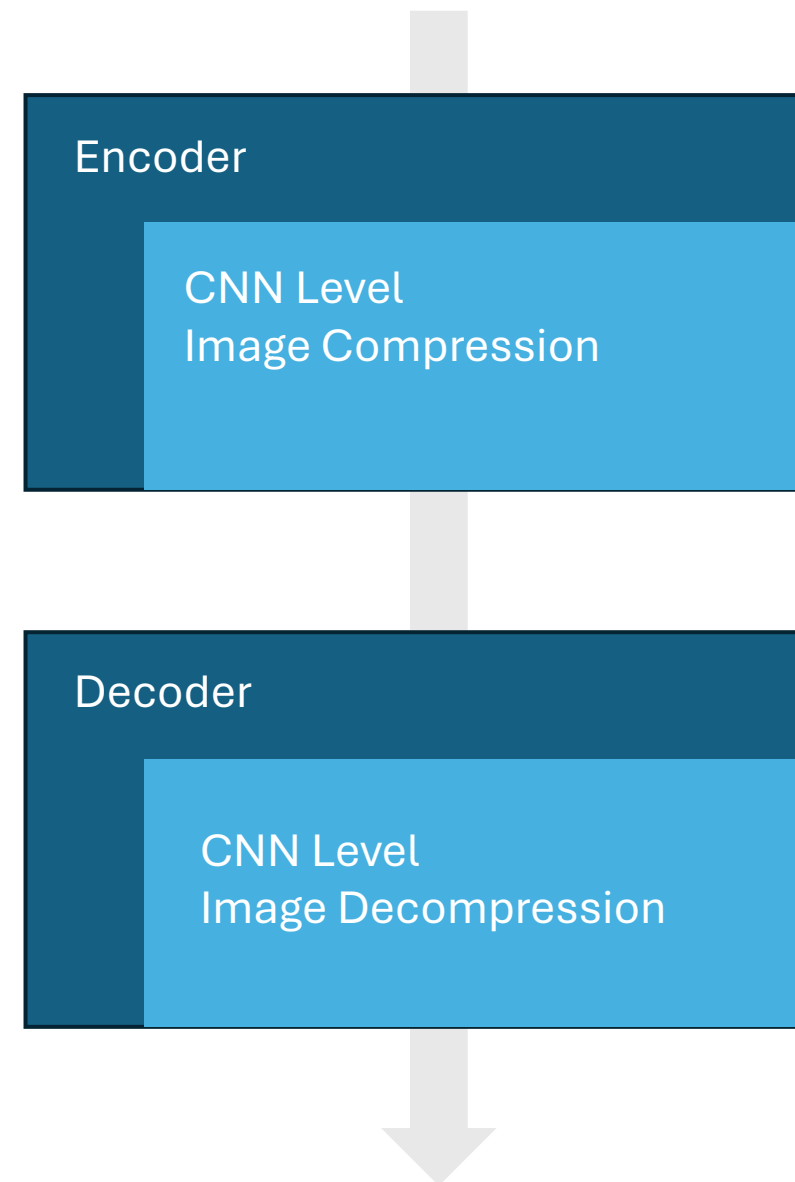
Training Flow: Encoder and Decoder

Encoder compresses data while Decoder reconstructs original input.



Dataset : Normal Data Only

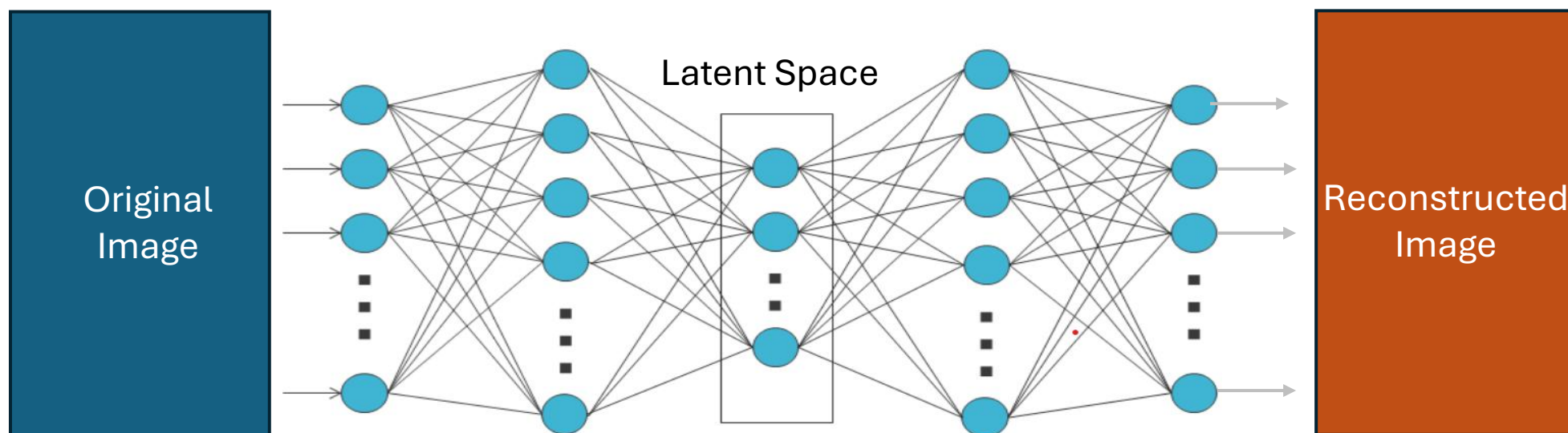
Trained exclusively on normal examples to identify abnormalities.



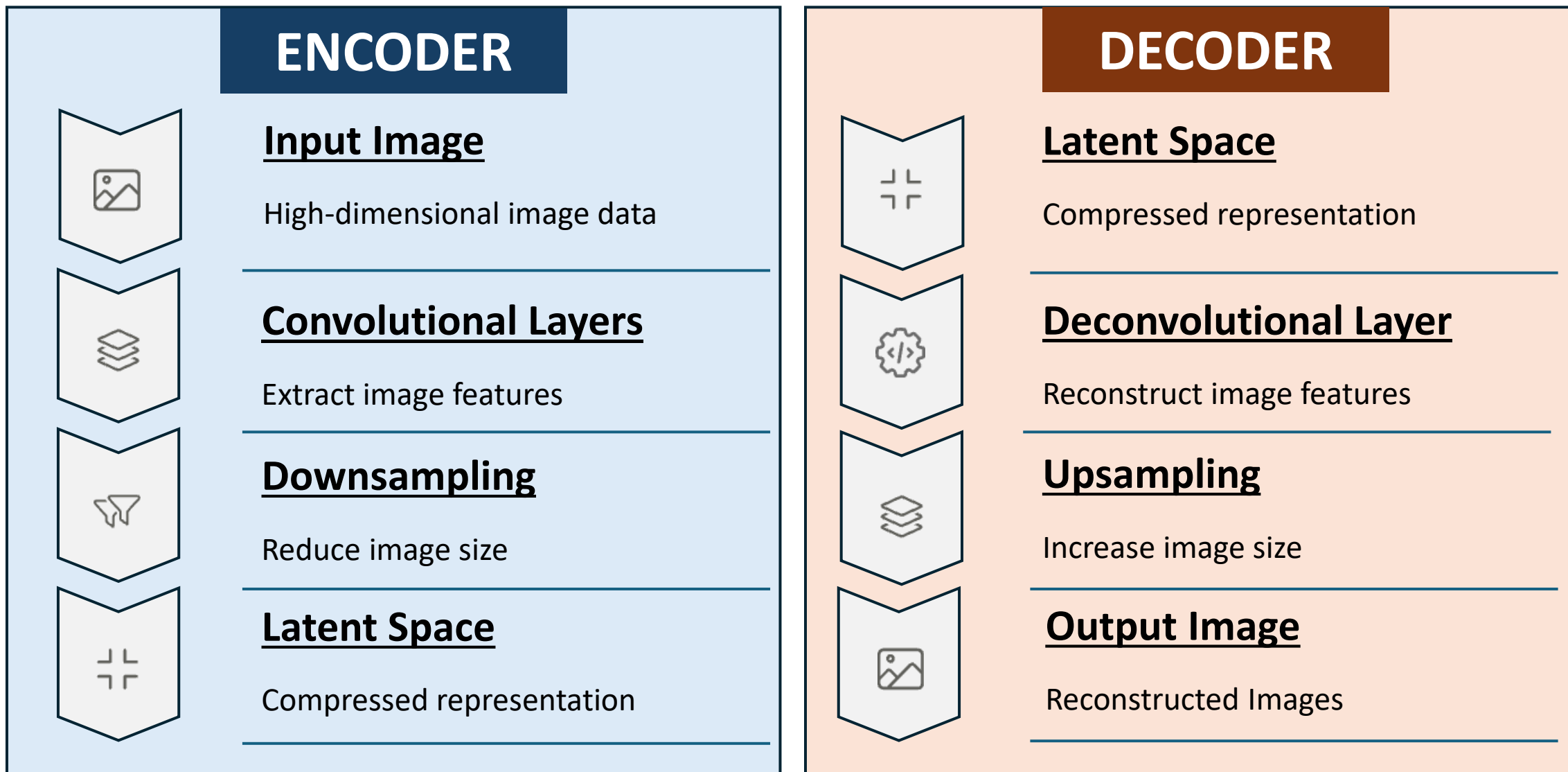
Autoencoder - How to detect Anomaly?

Given an input image $x \in \mathbb{R}^{H \times W \times C}$:

- Encoder maps $x \rightarrow z \in \mathbb{R}^d$, where $d \ll HWC$
- Decoder maps $z \rightarrow \hat{x} \in \mathbb{R}^{H \times W \times C}$
- Loss Function: $L(x, \hat{x}) = \|x - \hat{x}\|^2$
- Anomaly condition: $L(x, \hat{x}) > \text{threshold} \Rightarrow x$ is anomalous



Autoencoder - How to detect Anomaly?



Autoencoder - Real Analysis

```
self.encoder = nn.Sequential(
    nn.Conv2d(3, 32, 3, stride=2, padding=1), nn.ReLU(),
    nn.Conv2d(32, 16, 3, stride=2, padding=1), nn.ReLU(),
    nn.Conv2d(16, 8, 3, stride=2, padding=1), nn.ReLU()
)
```

Encoder

```
self.decoder = nn.Sequential(
    nn.ConvTranspose2d(8, 16, 3, stride=2, padding=1, output_padding=1), nn.ReLU(),
    nn.ConvTranspose2d(16, 32, 3, stride=2, padding=1, output_padding=1), nn.ReLU(),
    nn.ConvTranspose2d(32, 3, 3, stride=2, padding=1, output_padding=1), nn.Sigmoid()
)
```

Decoder

Build The CNN Layer of Autoencoder

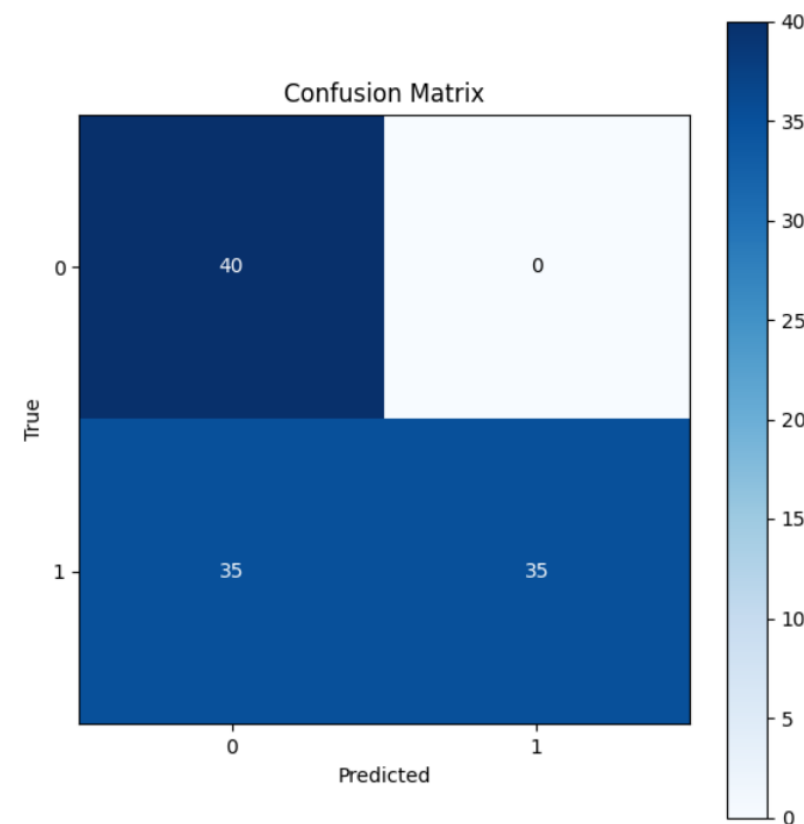
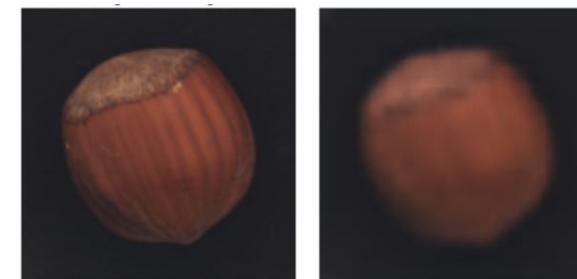
- Each layer in the encoder learns to extract and transform image features. For example, the first layer converts the RGB input (3 channels) into a 32-channel feature map. The second layer compresses and transforms the 32-channel feature map into 16 channels, capturing higher-level features with fewer channels.
- The decoder consists of three transposed convolutional layers that progressively upsample the compressed feature maps back to the original image size. Each layer increases spatial resolution and reduces the abstraction level, reconstructing the image from the encoded representation.

Real Analysis – Hazelnut Dataset

- The confusion matrix shows that the model correctly classified all 40 normal samples (class 0) with no false positives.
- However, for the 70 anomaly samples (class 1), it only correctly identified 35, misclassifying the other 35 as normal..
- The model shows a strong trade-off between recall and precision — while precision is high, the low recall significantly impacts the F1-score performance.

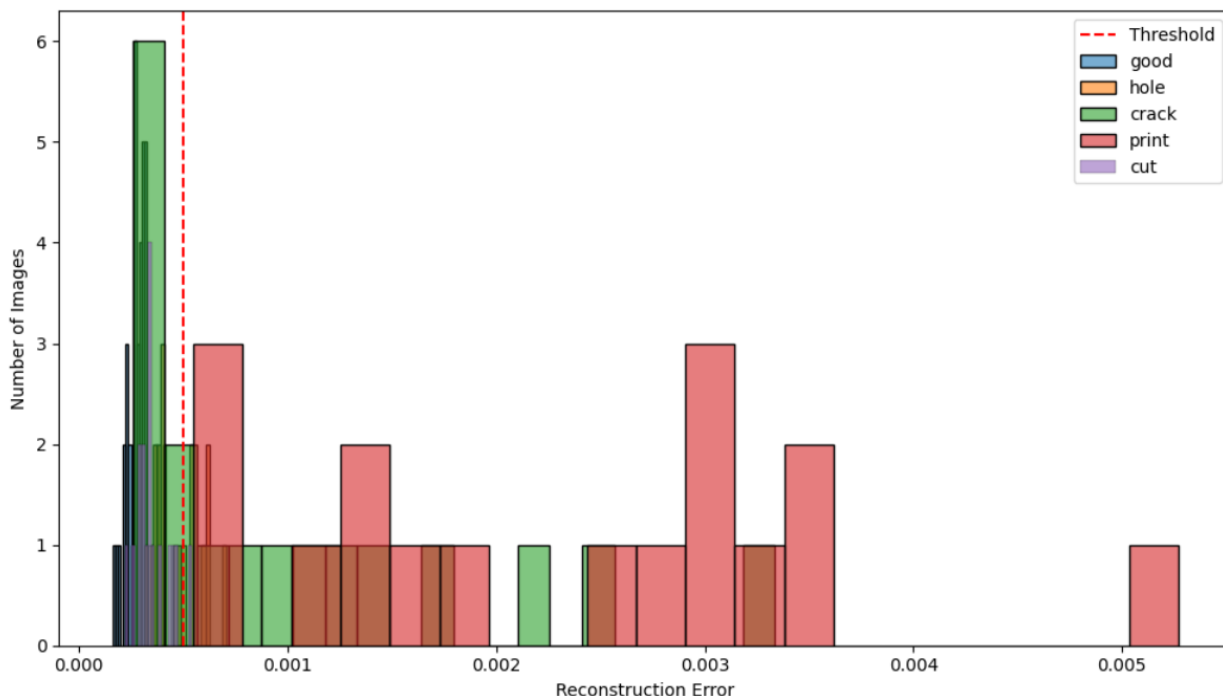
Accuracy	F1 score	Precision	Recall
0.7	0.69	0.84	0.70

Training Sample : 391

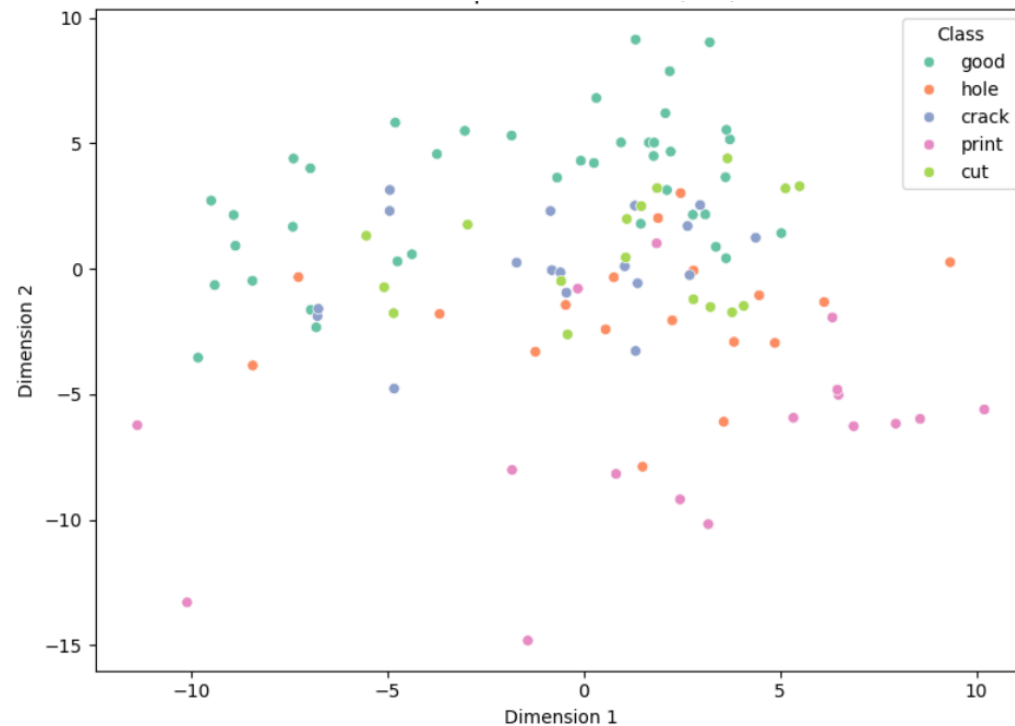


Real Analysis – Hazelnut Dataset

Reconstruction Error Distribution



Latent Space Visualization (PCA)

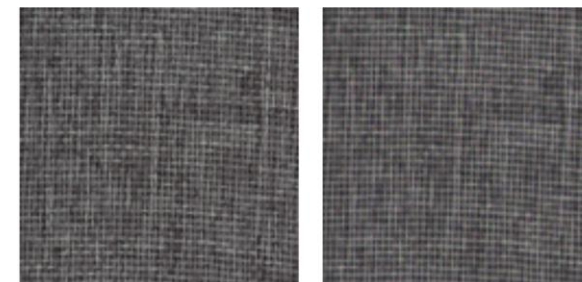


- Left Chart: Most good samples have reconstruction errors below this line, while defective types like cut and print tend to exceed the threshold, showing the model can differentiate anomalies via reconstruction loss.
- Right Chart: After PCA projection, we observe that certain classes like cut are more separable in the latent space, while others (e.g., hole, crack) are closer to the normal samples.

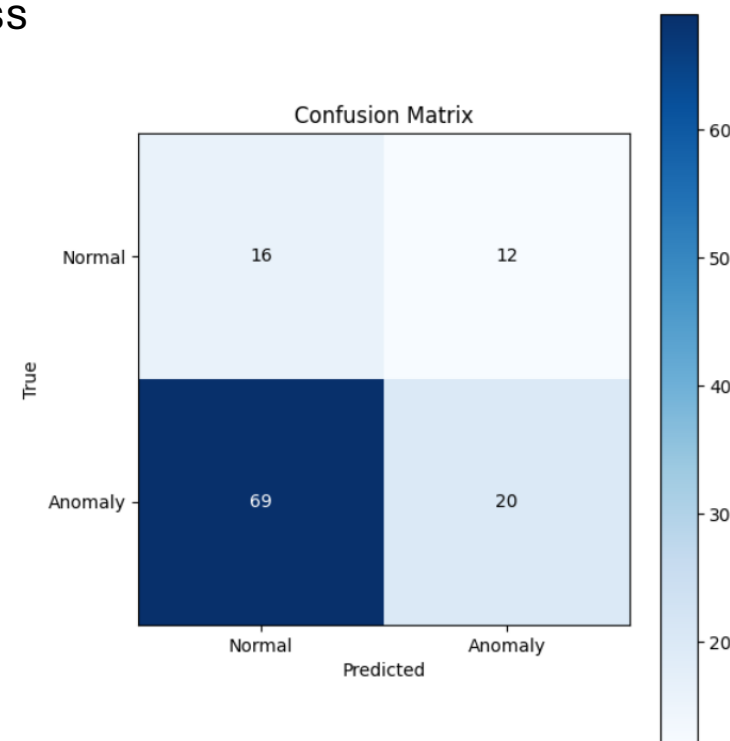
Real Analysis – Carpet Dataset

- This dataset is clearly more challenging than the Hazelnut dataset. The visual defects in carpet images are subtler,
- Despite tuning the CNN structure, the model achieves an F1-score below 0.40. We believe the main limitation is the small training samples (280).
- In addition, the anomalies in the carpet dataset are more subtle and less distinguishable, leading to poor recall and many false negatives.

Training Sample : 280

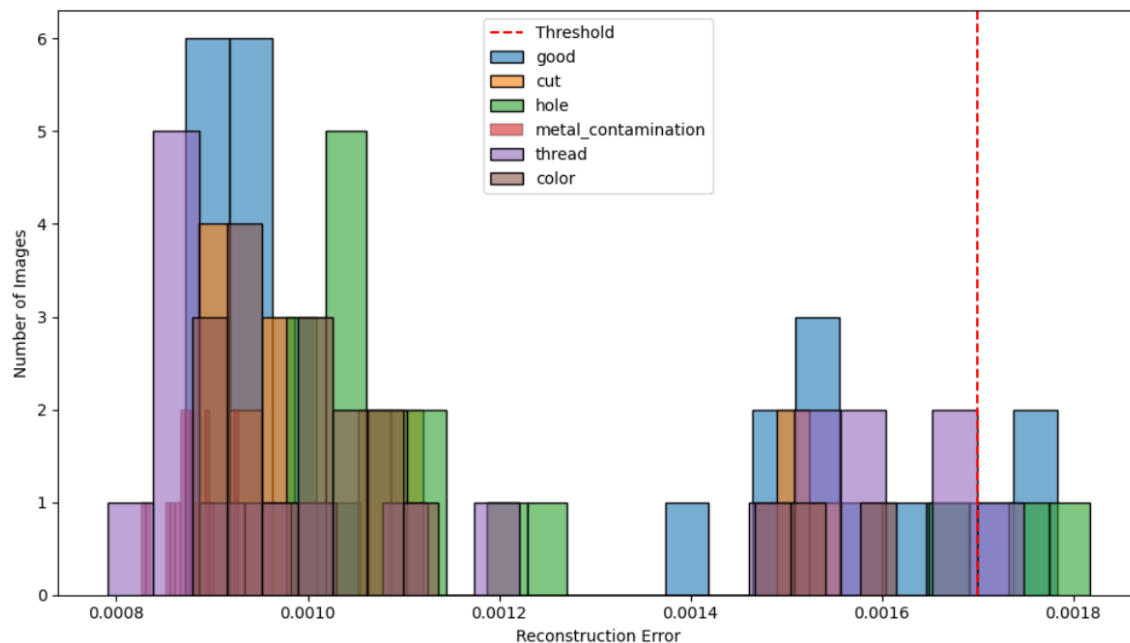


Accuracy	F1 score	Precision	Recall
0.32	0.33	0.53	0.32

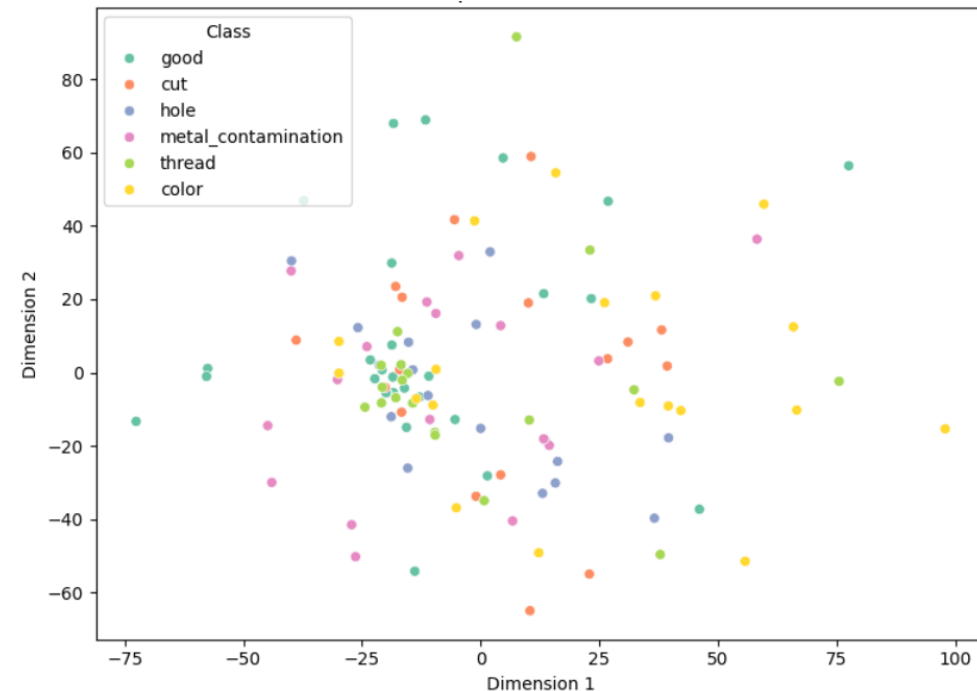


Real Analysis – Carpet Dataset

Reconstruction Error Distribution



Latent Space Visualization (PCA)



- Left Chart : In this case , no matter where the threshold is , it's still hard to distinguish the different category
- Right Chart: Compared with Hazelnut, we can see all the points mixed together.
- In this dataset, because the training-set is too small. The model can't learn the detail to detect anomaly well.

Autoencoder – Pros and Cons



Advantages

1. Only requires “normal” data (unsupervised learning).
2. Can handle high-dimensional datasets.
3. Flexible design the CNN layers for different datasets



Disadvantages

1. Highly time consuming for training
2. Require huge amount of training data for good detection
3. Not easy to find the suitable hyperparameter in the layers.

Conclusion :

- Anomaly with significant change of color and shape are easier captured by Autoencoder model
- However, in this project with small dataset, training set is too small and autoencoder is too time consuming.
- So , we are trying more model that can have good performance and improve the time efficiency

DBSCAN

Density-Based Spatial Clustering of Applications with Noise

Key Concept

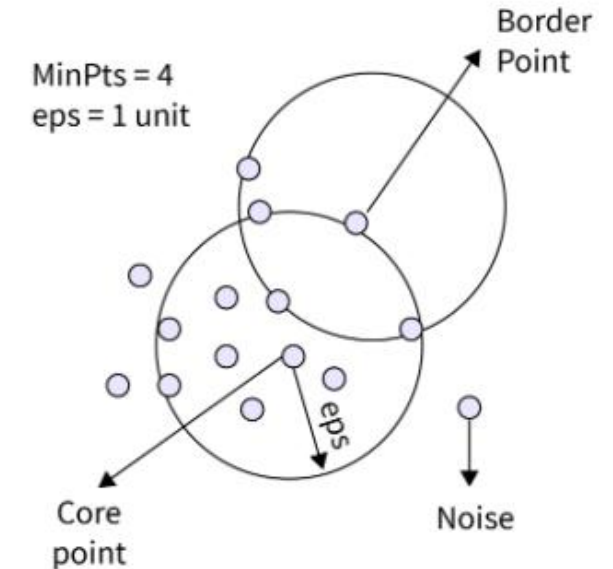
- DBSCAN is an unsupervised clustering algorithm that groups data points based on density and identifies outliers as noise without needing to predefine the number of clusters.
- Clusters are defined as regions of high point density separated by regions of low density.
- Unlike K-Means, DBSCAN can discover clusters of arbitrary shapes and automatically detect anomalies.

Algorithm

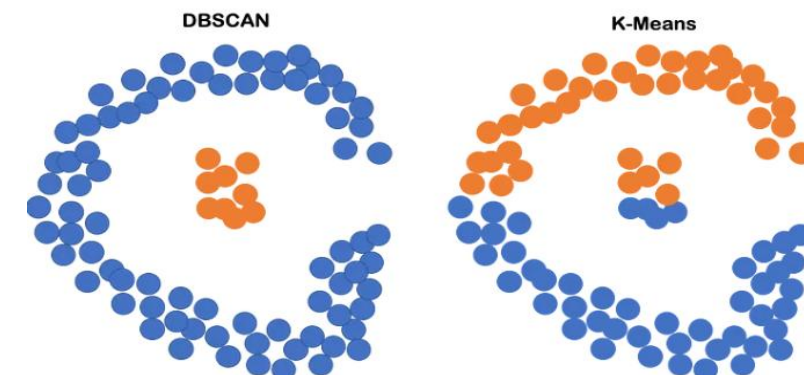
1. For each point, count how many neighbors exist within a given radius (ϵ).
2. If a point has at least **min_samples** neighbors $\rightarrow$ **core point**.
3. Expand clusters by connecting core points and their reachable neighbors.
4. Points that are not reachable from any core are labeled as **outliers (noise)**.

Math

- **ϵ (epsilon):** Neighborhood radius
- **min_samples:** Minimum number of points in ϵ -neighborhood to be a core point
- For point p , if $\{q \in D \mid \text{distance}(p, q) \leq \epsilon\} \geq \text{min_samples} \rightarrow p$ is a core point
- Clusters = sets of density-connected core points
- Noise = points not density-reachable from any core



- ϵ (epsilon): Neighborhood radius
- min_samples: Minimum points required to form a dense region
- Noise: Points not density-reachable from any core



Model Test & Result

Feature extraction

- Pretrained CNN(ResNet18)

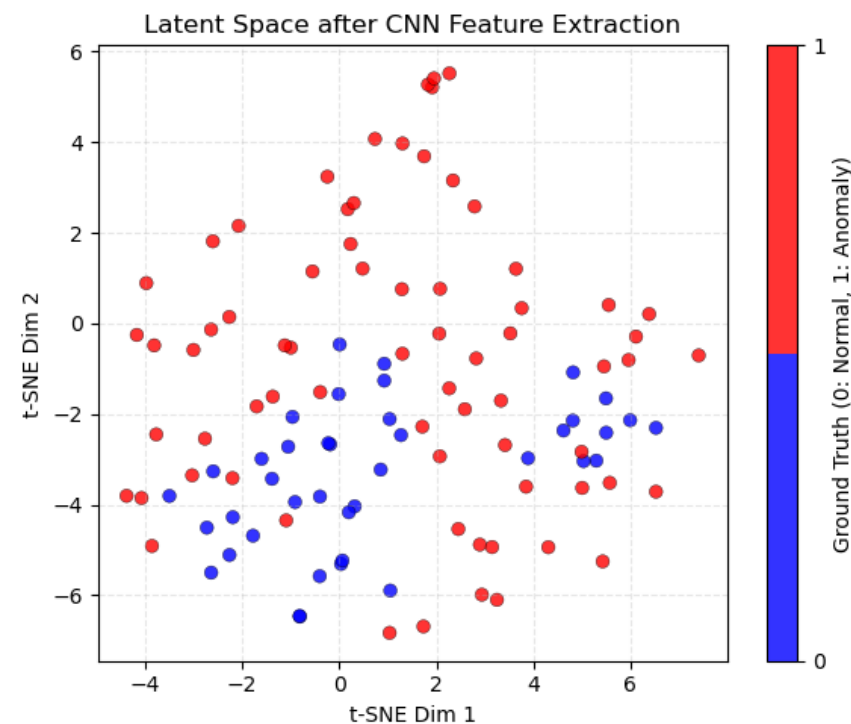
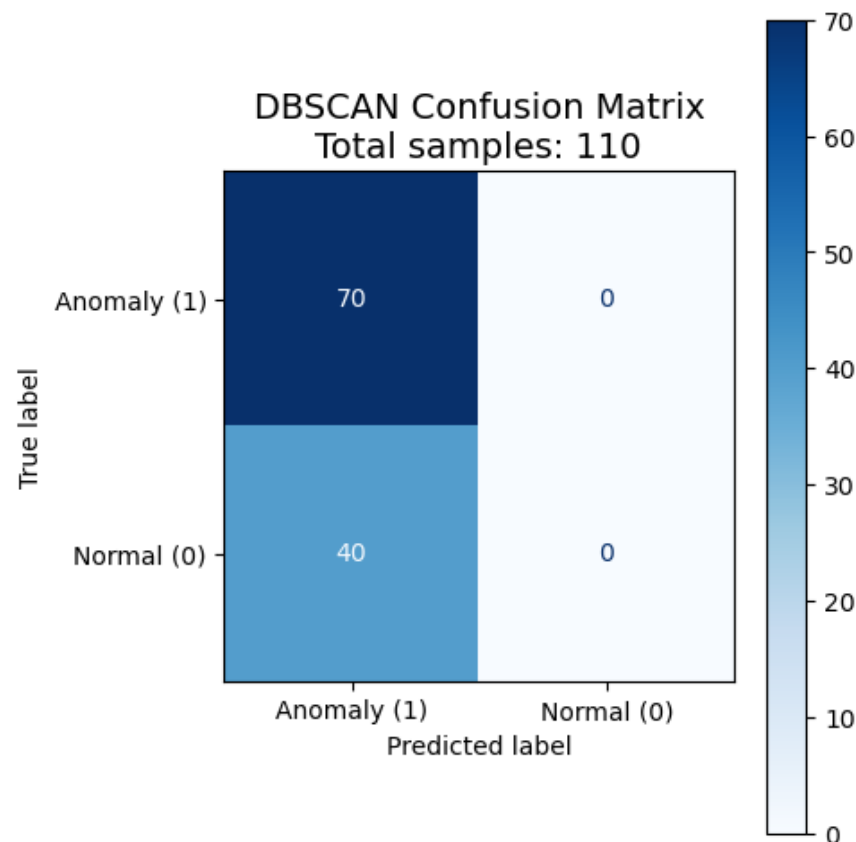
Anomaly Detection

- DBSCAN training: unsupervised clustering with normal images
- Bayesian optimization: Optuna

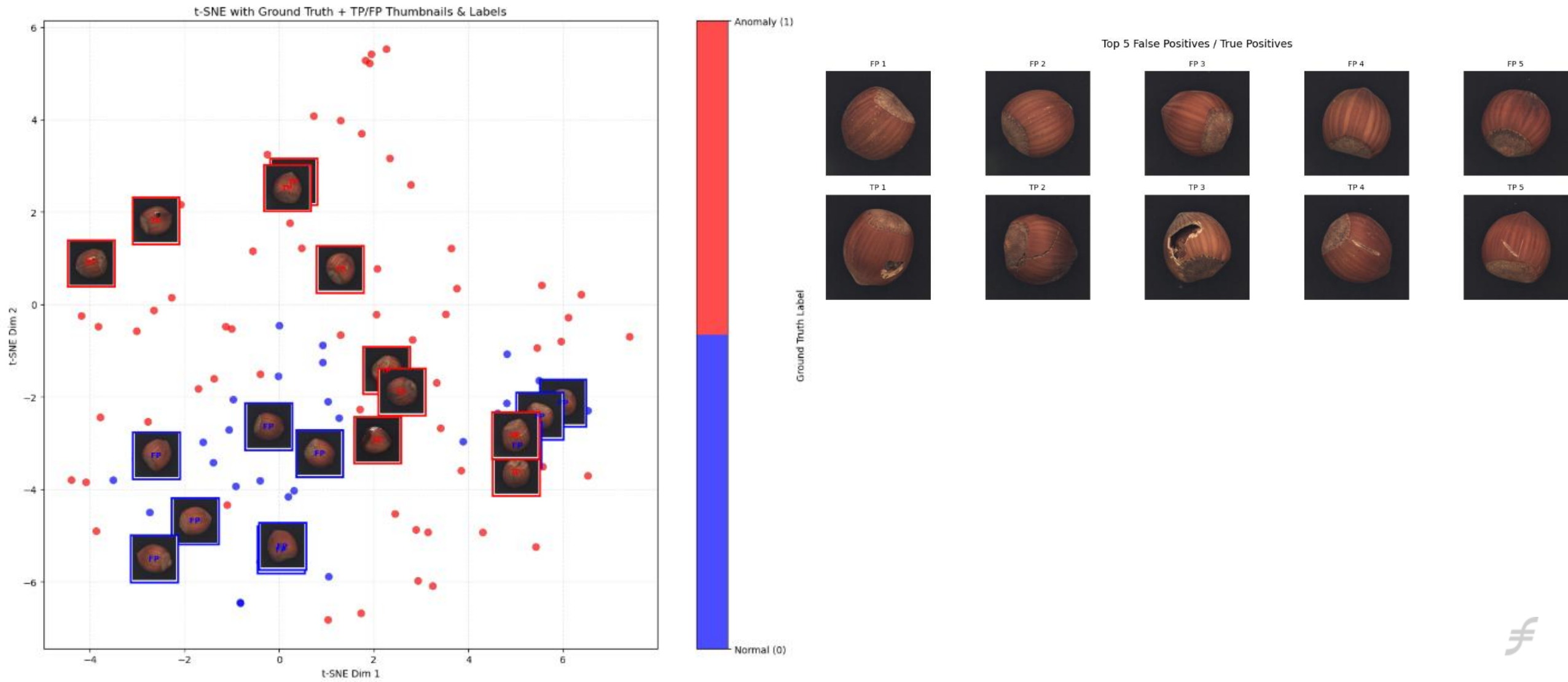
Performance Metrics

- F1 score, recall %, precision %

Model	eps	min_samples	Accuracy	Precision	Recall	F1 Score	ROC AUC	PR AUC	Support	Mean Score
Optuna Best	0.9367	20	0.6364	0.6364	1.0000	0.7778	0.9507	0.9747	110	9.7639



Model Test & Result



Model Test & Result

Result

- **0% normal recall**
all samples labeled as outliers across all parameter settings
- **Generic feature space**
Highly sensitive to feature quality—fails entirely on generic embeddings.
Fixed density threshold (eps) cannot adapt to complex, high-dimensional distributions.
- **No clear cluster separation**
Without fine-tuning or domain adaptation, the feature manifold does not exhibit distinct density regions for normal vs. anomalous samples.



Potential Improvement

- **Tailored latent space(feature extraction):** Auto Encoder
learns a tailored latent space where defects yield high reconstruction or embedding errors.
- **Dimensional reduction to core features:** PCA, UMAP
- **Non-linear embedding:** HDBSCAN
to capture non-linear structure and variable density before clustering.
- **Two-stage detection**
Use image-level filtering followed by patch-level localization (e.g., Conditional Random Fields) to refine false positives.

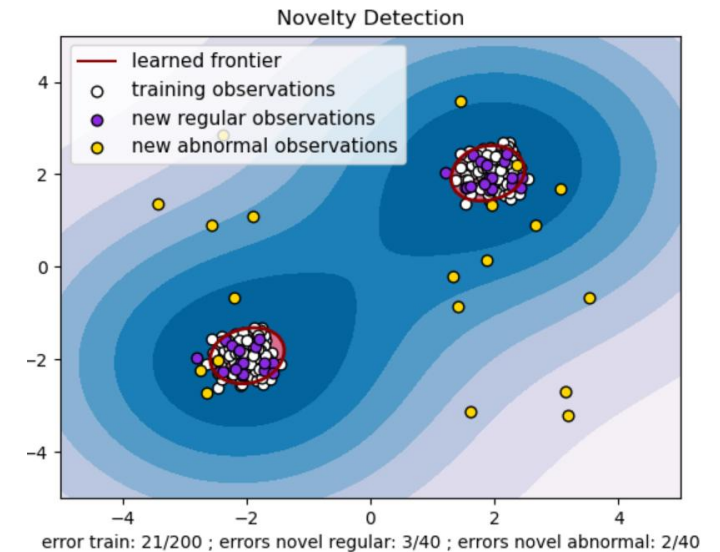
Method	Feature Extraction	Dimensional Reduction	F1 Score	Precision	Recall	Accuracy	PR AUC	ROC AUC
DBSCAN	CNN		0.78	0.64	1.00	0.64	0.97	0.95
	CNN	PCA+UMAP	0.55	0.88	0.40	0.58	0.80	0.68
	Auto-encoder		0.50	0.41	0.64	0.64	0.82	0.50
HDBSCAN	CNN		0.80	0.67	1.00	0.68	0.67	0.56
	CNN	PCA+UMAP	0.82	0.80	0.84	0.76	0.91	0.83

ONE-CLASS SVM

One-class SVM

Key Concept

- One-Class SVM is an **unsupervised** model used for anomaly detection using Support Vector Machines (SVM).
- One-Class SVM assumes all training samples belong to the “normal” class and learns a **decision boundary** that encloses most of these normal data in high-dimensional space.
- It **maximizes the margin between data points and the origin in feature space**. (The origin stands for a “dummy” anomaly class.)
- Any point outside this boundary is classified as an anomaly.



Source: scikit learn

One-class SVM

Math

- **Decision function:** Points with $f(x) \geq 0$ are labeled “normal”, and those with $f(x) < 0$ “anomalous”.

$$f(x) = \text{sign}(w^T \phi(x) + b)$$

- **Learning objective:** Maximize the Margin while Minimizing Misclassified Samples

$$\min_{w,b} \frac{1}{2} \|w\|^2 + \frac{1}{\nu n} \sum_{i=1}^n \max(0, -(w^T \phi(x_i) + b))$$

- ✓ The first term aims to maximize the margin between the “normal” region and the origin, which helps the model better distinguish between normal and abnormal samples.
- ✓ The second term controls how many training points can lie outside the learned boundary.

Meaning of Each Component

- x : Training sample set; w : Weight vector of the decision hyperplane, which used to determine the boundary;
- b : Bias term of the decision hyperplane, which determines the offset of the boundary; n : Total number of training samples.
- $\nu \in (0,1]$: Hyperparameter of One-Class SVM that controls the tolerance for misclassified samples (the maximum error rate).
- **Larger $\nu \rightarrow$ more points can lie outside the boundary $\rightarrow$ more anomalies caught but also more normals mis-flagged.**

One-class SVM

Algorithm

1. Kernel selection: Choose appropriate kernel: RBF, Polynomial, or Linear.
2. Parameter tuning: Optimize ν and RBF gamma parameters for best performance.
3. Training: Train exclusively on normal data samples.
4. Decision function: Apply decision function where sign determines anomaly status.

Pros

- Only requires “normal” data (unsupervised learning).
- Can handle high-dimensional datasets.
- Flexible kernel selection for different data types.

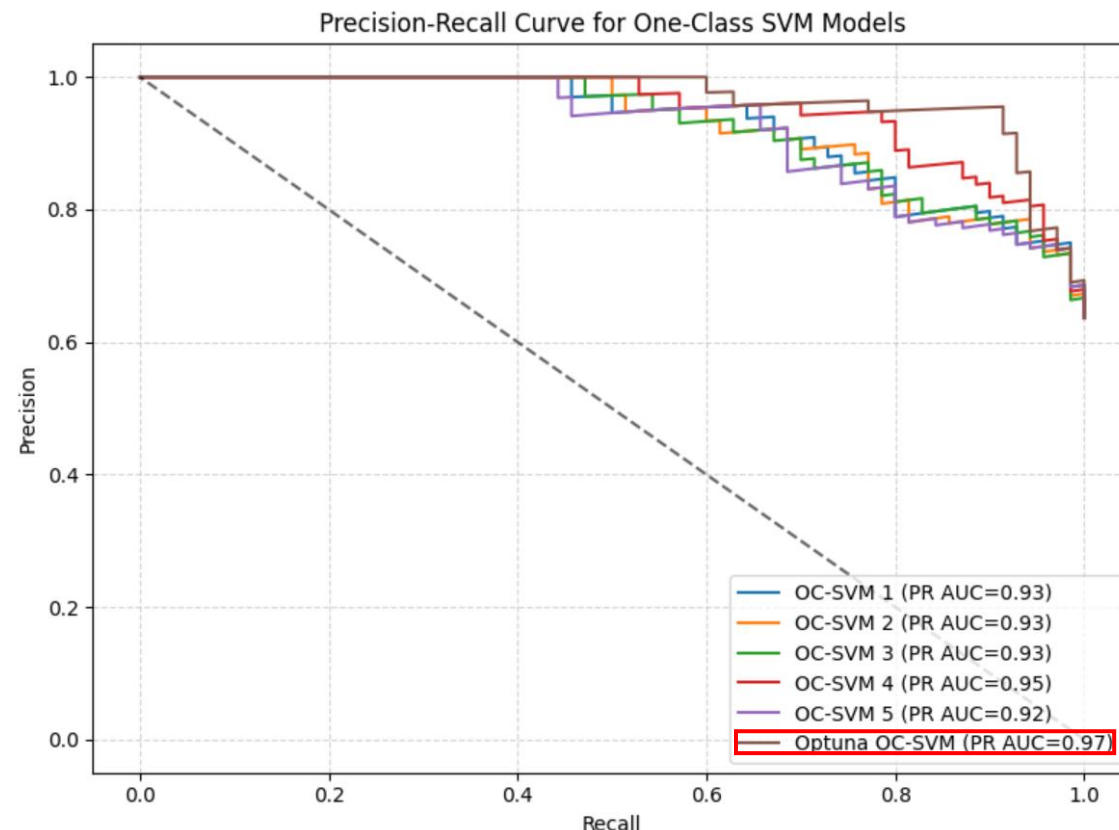
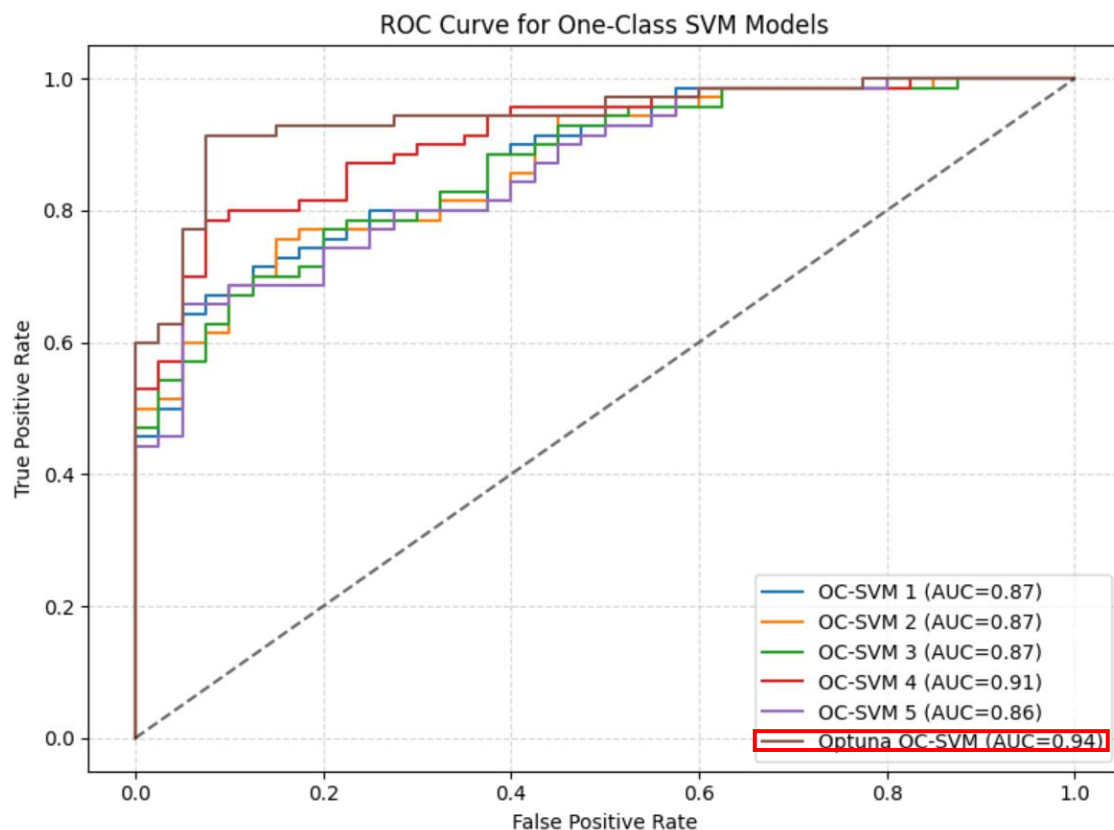
Cons

- Highly sensitive to parameter settings; requires careful tuning.
- Assumes a single anomaly distribution; struggles to detect complex anomalies.
- Computational cost grows quickly with sample size and feature dimensionality.

One-class SVM Model TEST

Dataset: Hazelnut (from MVTec AD)

- **Step 1:** Feature extraction – using pretrained resnet18 model (CNN)
- **Step 2:** Kernel selection – using RBF kernel (Most common choice for non-linear data separation. Good performance for complex data relationships.)
- **Step 3:** Parameter tuning – **using F1 as primary score**. 2 parameter include (1) Nu: Controls error acceptance (0 to 1); (2) RBF Gamma: Defines Influence radius of training examples.

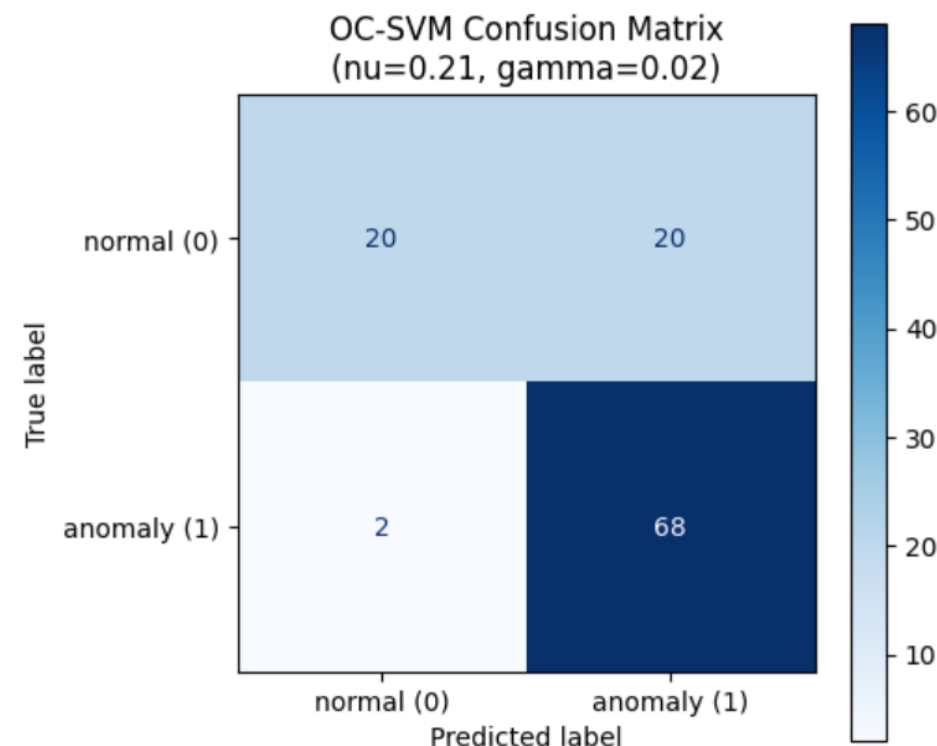


One-class SVM Model TEST

Dataset: Hazelnut (from MVTec AD)

- Step 4: Model results on test set

PR AUC	ROC AUC	F1 score	Precision	Recall
0.97	0.94	0.86	0.77	0.97



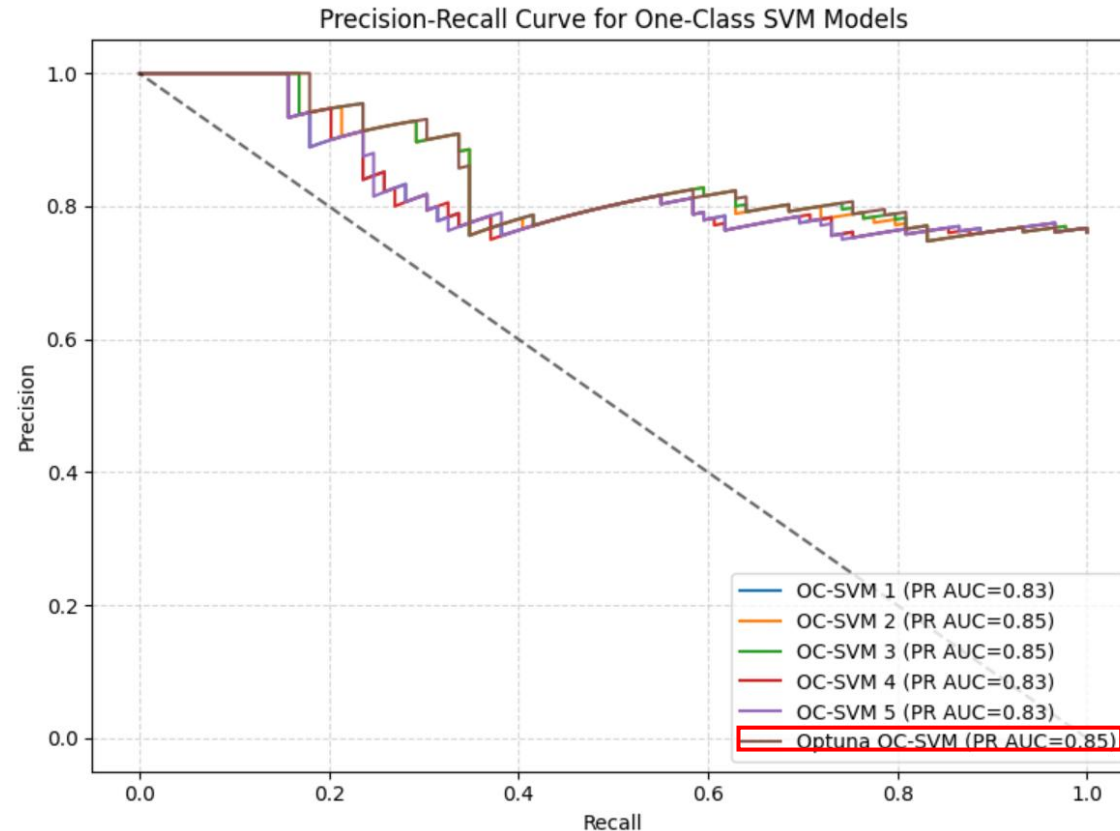
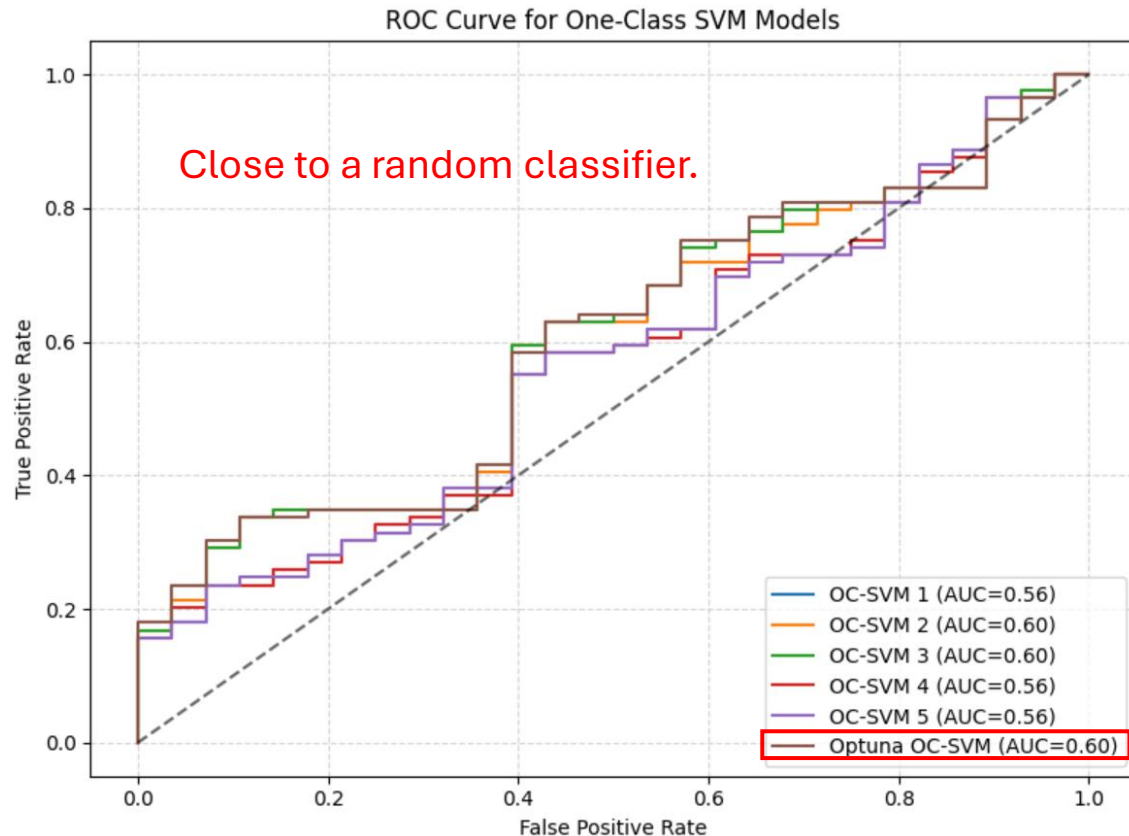
Takeaway:

- The parameter setting (1) $\nu = 0.21$ allows the model to exclude roughly the top 21% of points that deviate most from the normal distribution, while still using enough support vectors to accurately shape the decision boundary, (2) $\gamma = 0.02$ keeps the RBF kernel relatively wide, preventing the boundary from over-twisting around noise.
- The model is very good at detecting anomalies (high anomaly recall of 97% and PR AUC of 0.97).**
- It is somewhat over-cautious on normals, mis-classifying a fair number of them as anomalies (normal recall only 50%).**

One-class SVM Model TEST

Dataset: Carpet (from MVTec AD)

- **Step 1:** Feature extraction – using pretrained resnet18 model (CNN)
- **Step 2:** Kernel selection – using RBF kernel (Most common choice for non-linear data separation. Good performance for complex data relationships.)
- **Step 3:** Parameter tuning – **using F1 as primary score**. 2 parameter include (1) Nu: Controls error acceptance (0 to 1); (2) RBF Gamma: Defines Influence radius of training examples.

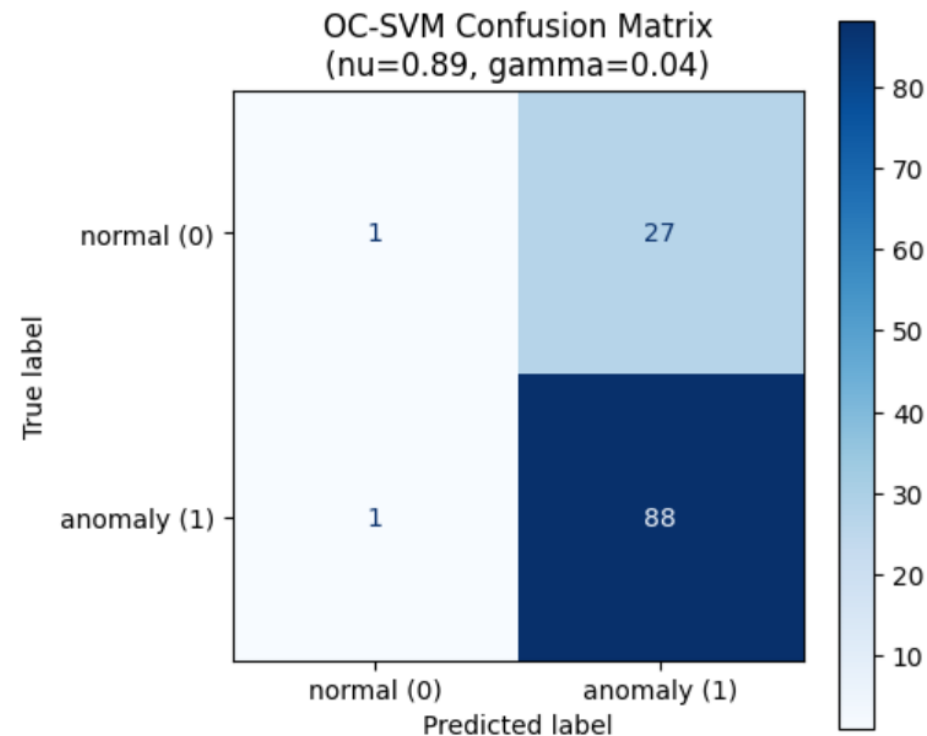


One-class SVM Model TEST

Dataset: Carpet (from MVTec AD)

- Step 4: Model results on test set

PR AUC	ROC AUC	F1 score	Precision	Recall
0.85	0.60	0.86	0.77	0.99



Takeaway:

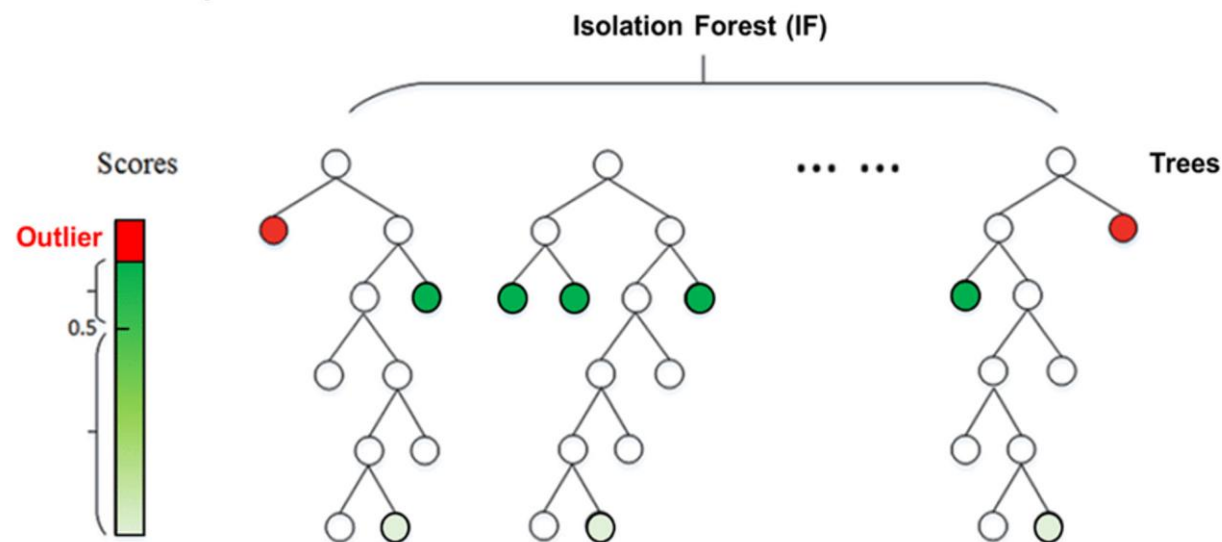
- One-Class SVM struggles with complex, high-dimensional anomaly patterns**, especially when the normal/anomaly boundary is intricate—high ν and γ simply carve out almost the entire space as “anomaly”.
- Dataset-specific behavior:** On Hazelnut (object data), One-Class SVM still achieved a strong ROC AUC (0.94) and PR AUC (0.97), with moderate normal recall (50%). In contrast, on Carpet (material data), ROC AUC dropped to 0.60, PR AUC to 0.85, and normal recall collapsed to 4%, showing that it cannot model the highly variable texture patterns of carpet.

ISOLATION FOREST



What is an Isolation Forest?

Isolation Forest (iForest) is an **unsupervised machine learning algorithm** specifically designed for **anomaly detection**. It identifies data points that are "few and different" — which is the core intuition behind anomalies.



Source: Medium.



Methodology: Tree-Based Ensemble

Partitions data using randomly generated decision trees to isolate anomalies.



Target: Anomaly Isolation

Detects outliers by how easily data points can be separated. Anomalies require fewer splits / shorter paths.



Dataset: Unlabeled Data

Designed to work without labeled data, especially in scenarios where you don't know what an anomaly looks like.

iForest Algorithm

1. Subsample Data

- Randomly sample a subset of the data (max_samples) for each tree.

2. Build an Isolation Tree

- For each tree:
 - Randomly choose a feature.
 - Randomly pick a split value between the min and max of that feature.
- Split the data and recursively repeat for each branch.
- Stop if:
 - The instance is isolated, or
 - A maximum depth is reached (default: $\lceil \log_2(\text{sample size}) \rceil$).
- This process results in a binary tree where anomalies tend to be isolated quickly (closer to the root).

3. Calculate Path Length

- For a data point x , traverse each tree and count the number of splits (path length $h(x)$) needed to isolate x .



iForest Algorithm

- Compute the average path length at which point x gets isolated across all trees in the forest:

$$E(h(x)) = \frac{1}{t} \sum_{i=1}^t h_i(x)$$

Where:

- t – number of isolation trees
- $h_i(x)$ – path length of x in the i -th tree

4. Compute Anomaly Score

- Use this formula to get an anomaly score for each data point:

$$s(x, n) = 2^{\frac{-E(h(x))}{c(n)}}$$

Where:

- $c(n)$ is the normalization factor / theoretical average path length of a normal point in a sample of size n , where:

$$c(n) \approx 2 \cdot (\ln(n-1) + \gamma) - \frac{2(n-1)}{n}$$

where $\gamma \approx 0.577$ (Euler's constant)

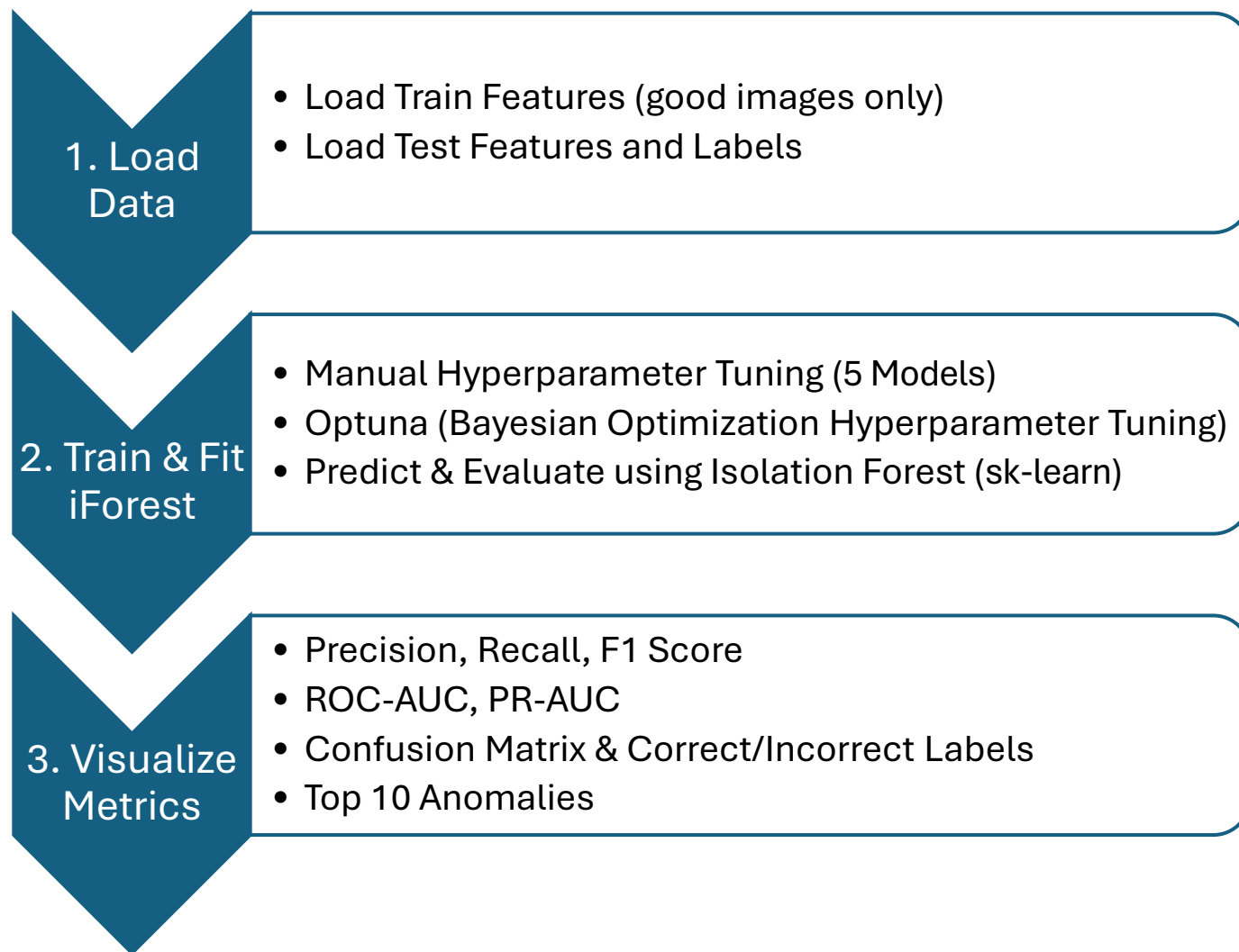
5. Make decisions

- Flag data points as anomalies if their score exceeds a certain threshold (e.g. contamination parameter).
- The closer the score $s(x, n)$ is to 1 $\rightarrow$ the more likely it's an **anomaly**.
- The closer the score is to 0 $\rightarrow$ the more likely it's **normal**.



Source: Medium.

iForest Workflow in Image Anomaly Detection

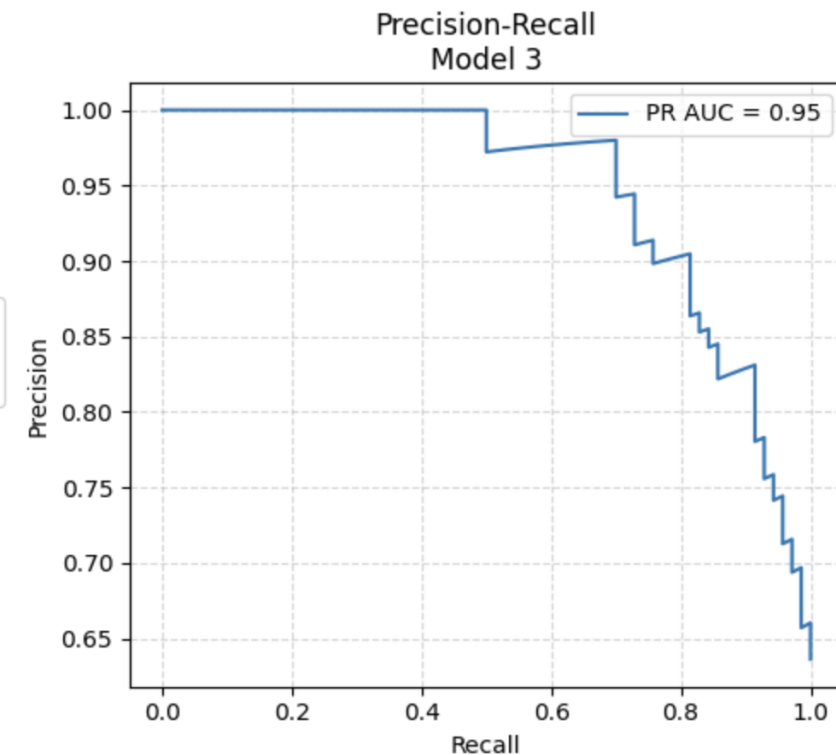
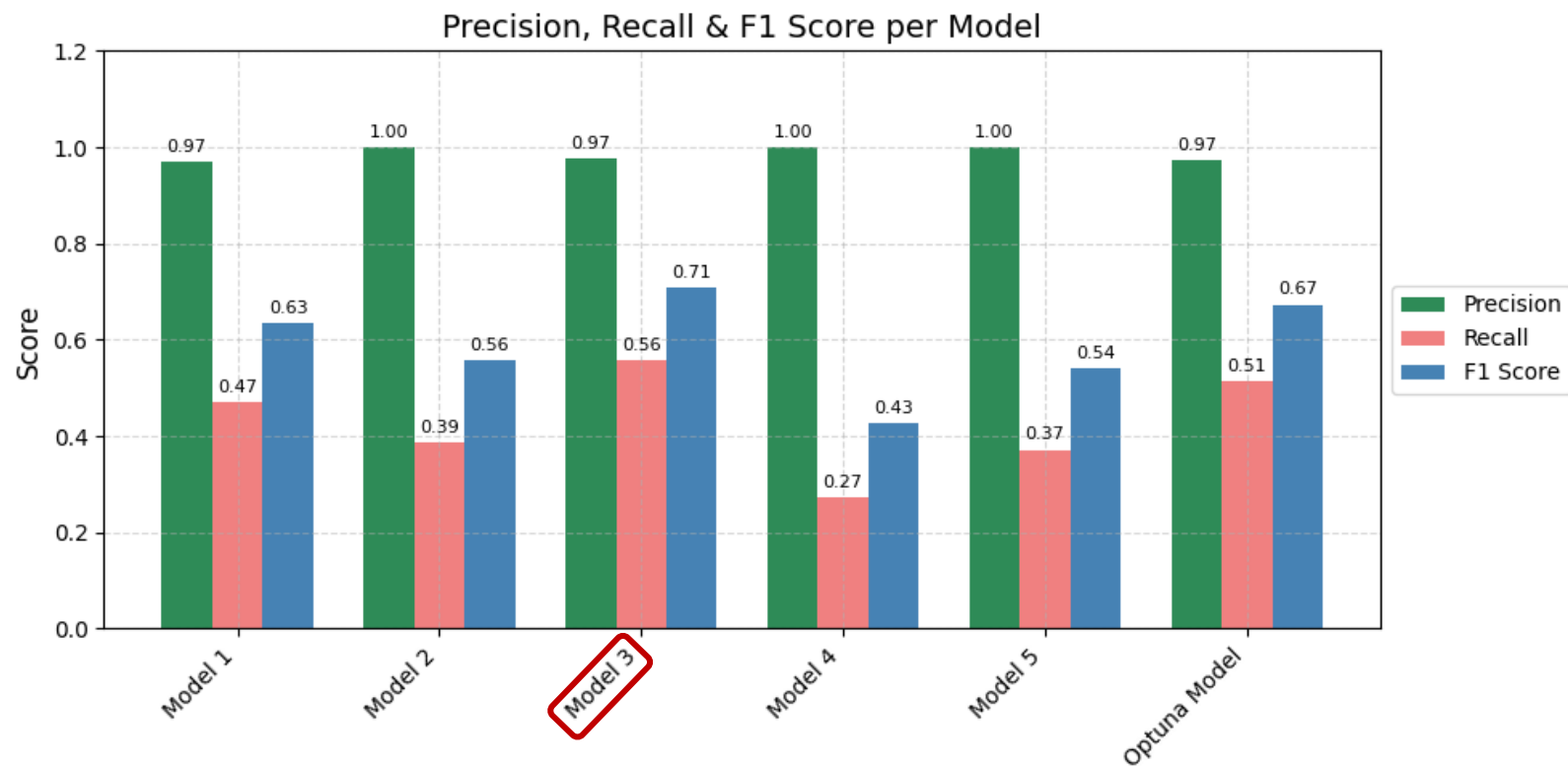


Main Parameters

- `n_estimators`
- `max_samples`
- `contamination`
- `max_features`
- `random_state`

Model Usage (Negated)	Metrics Scoring
Anomaly (+1)	Anomaly (1) - Positive class
Normal (-1)	Normal (0) - Negative class

iForest Hazelnut Results



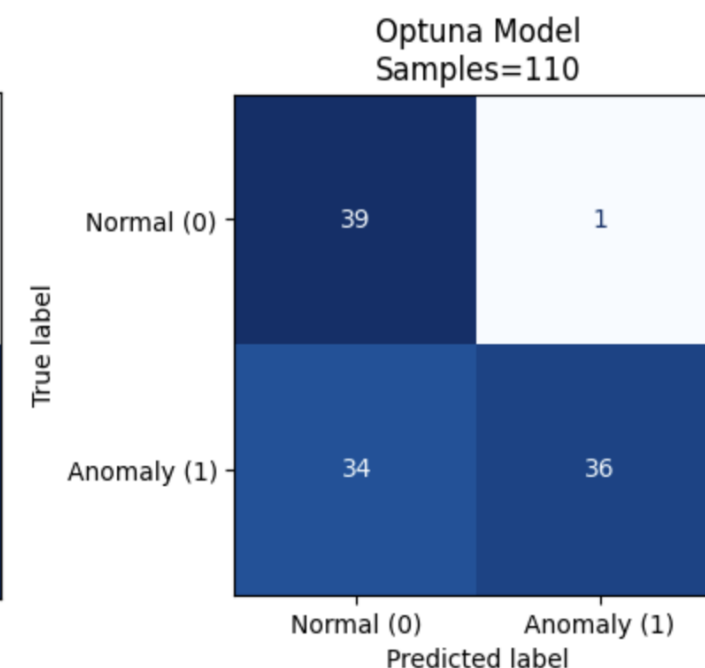
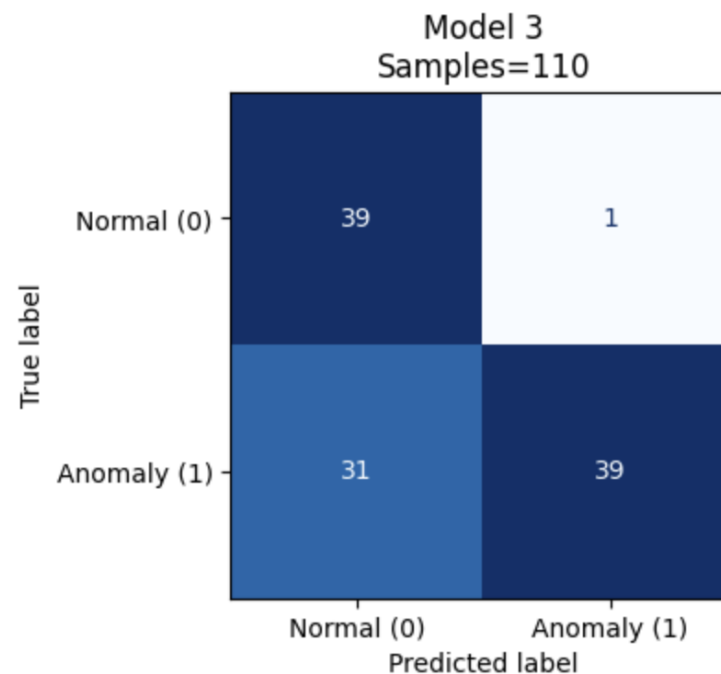
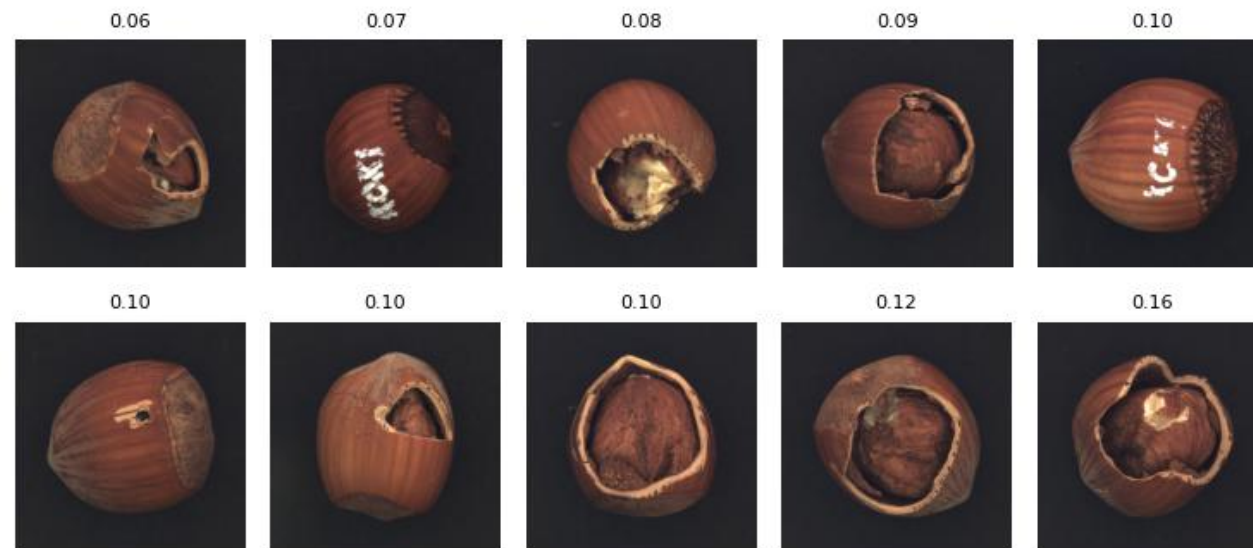
Best Model (w.r.t. F1): Model 3

PR AUC	ROC AUC	F1 score	Precision	Recall
0.95	0.90	0.71	0.97	0.56

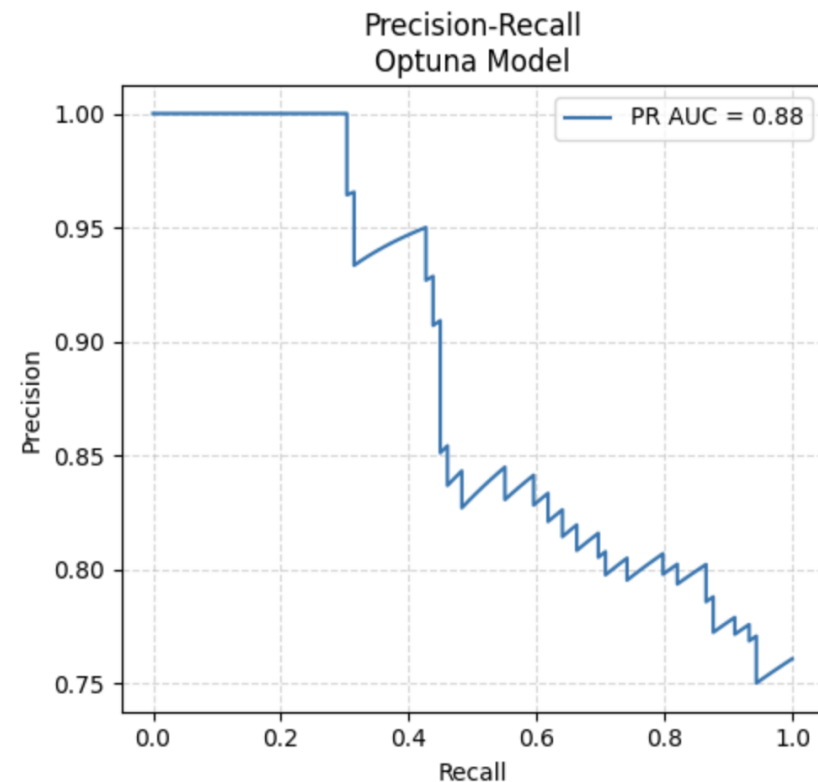
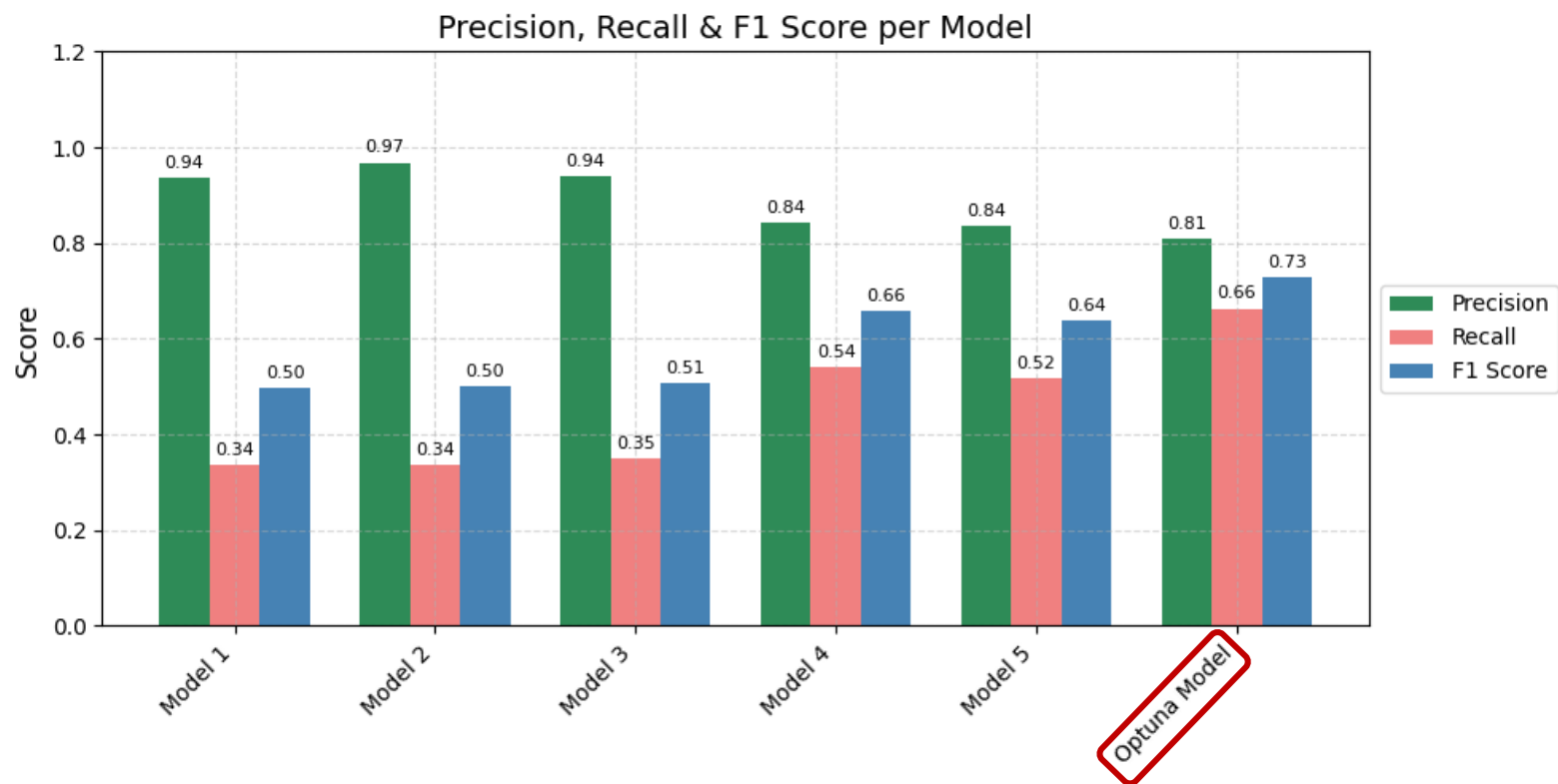
iForest Hazelnut Results

- **Moderate anomaly detection capability** (F1 Score: 0.71)
- **Slightly imbalanced PR trade-off**
 - Zero false alarms for normal images (high Precision)
 - But missed more than half of true anomalies (moderate Recall)
 - Possible fix: adjust the decision threshold
- **Conservative: Flags clear anomalies only (Crack and Hole images)**
 - Misclassified Print and Cut defects due to subtle variations or similarities with normals
- **Minimal model improvement with Optuna**
 - Models manually tuned yielded to better Recall results.

Top 10 Anomalies (Model 3)



iForest Carpet Results



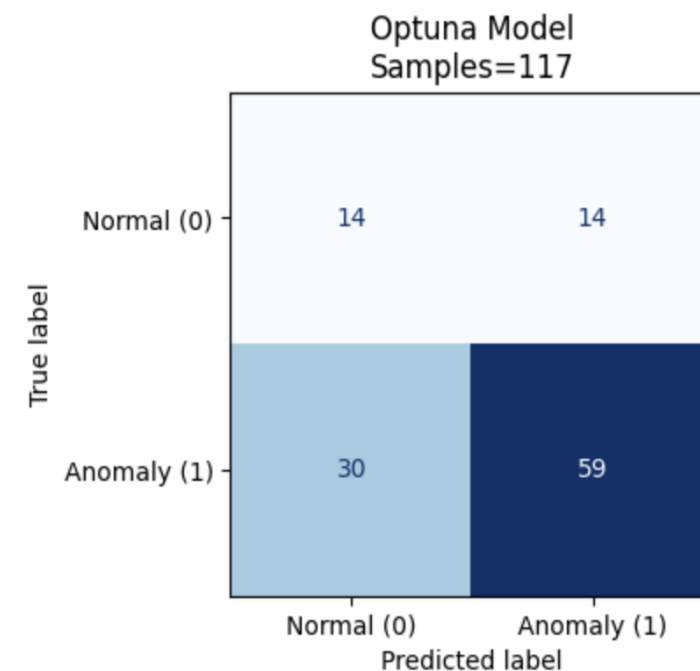
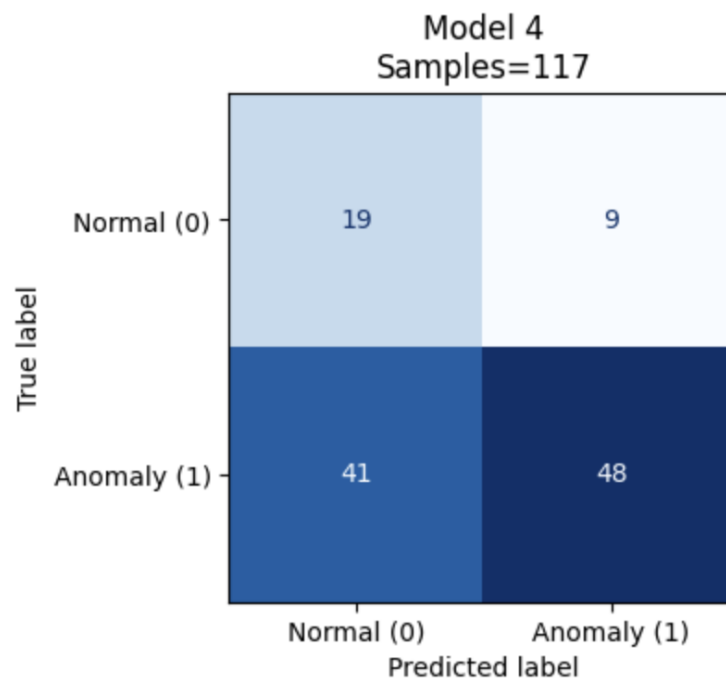
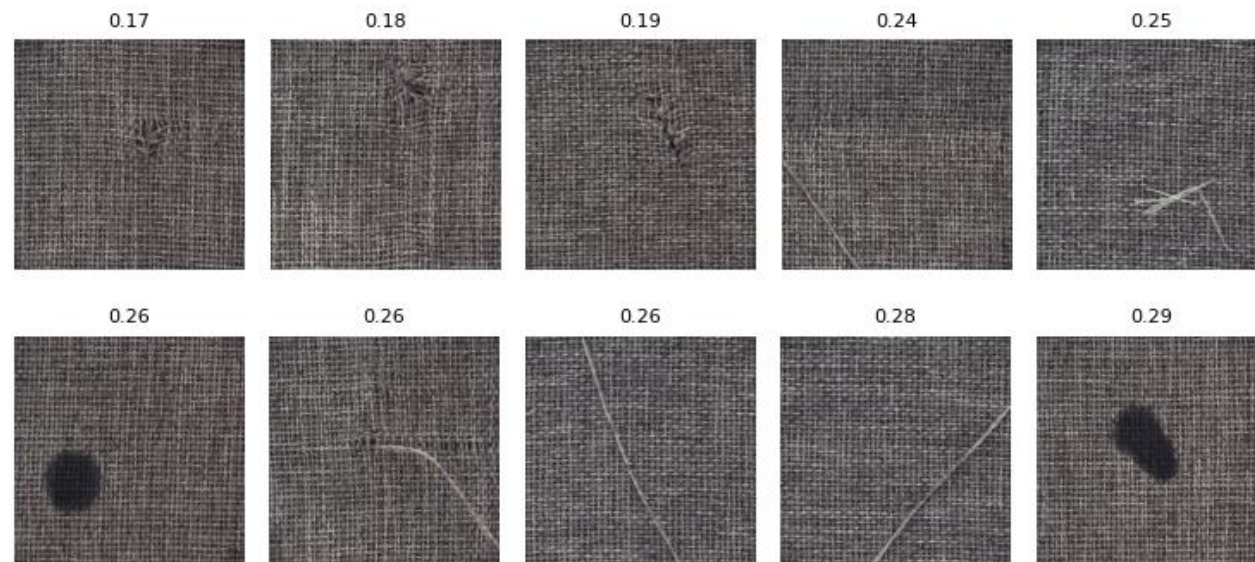
Best Model (w.r.t. F1) : Optuna Model

PR AUC	ROC AUC	F1 score	Precision	Recall
0.88	0.67	0.73	0.81	0.66

iForest Carpet Results

- **More balanced anomaly detection performance** (F1 Score: 0.73)
 - Some true anomalies still missed (Recall rate: 0.66)
 - Sometimes misclassify good images (Precision rate: 0.81)
- **Greater misclassifications due to a more challenging image dataset**
 - Carpet defects show subtle or inconsistent patterns
- **Faster isolation for Thread, Cut, Color defects**
 - Defects with strong deviations are more obvious
 - Hole and Metal contamination images are more subtle

Top 10 Anomalies (Optuna Model)



Isolation Forest conclusion



Advantages

1. Fits well without labeled anomalies
2. Fast and scalable, even on high-dimensional feature spaces
3. Easily interpretable Anomaly Scores
4. Effective on distinct outliers



Disadvantages

1. Struggles with subtle anomalies
2. Sensitive to hyperparameters (contamination, thresholds)
3. Cannot localize anomalies at pixel- or patch-level (global image scores only)

- **Conclusion:** Isolation Forest is a fast, unsupervised anomaly detection algorithm that works well when anomalies are **distinct and separable** from normal patterns.
 - **Dataset-specific:** High precision but low recall (0.56) on Hazelnut; improved recall (0.66) on Carpet at the cost of more false alarms
 - **Threshold tuning & ensemble:** Adjust decision-function threshold and employ a hybrid approach with PatchCore/Autoencoder for performance gains

PATCHCORE



What is Patchcore?



Methodology : Non-deep-learning-based anomaly detection

Uses pretrained CNN features and nearest-neighbor search without backpropagation or fine-tuning.



Target : Unsupervised anomaly detection in industrial visual inspection

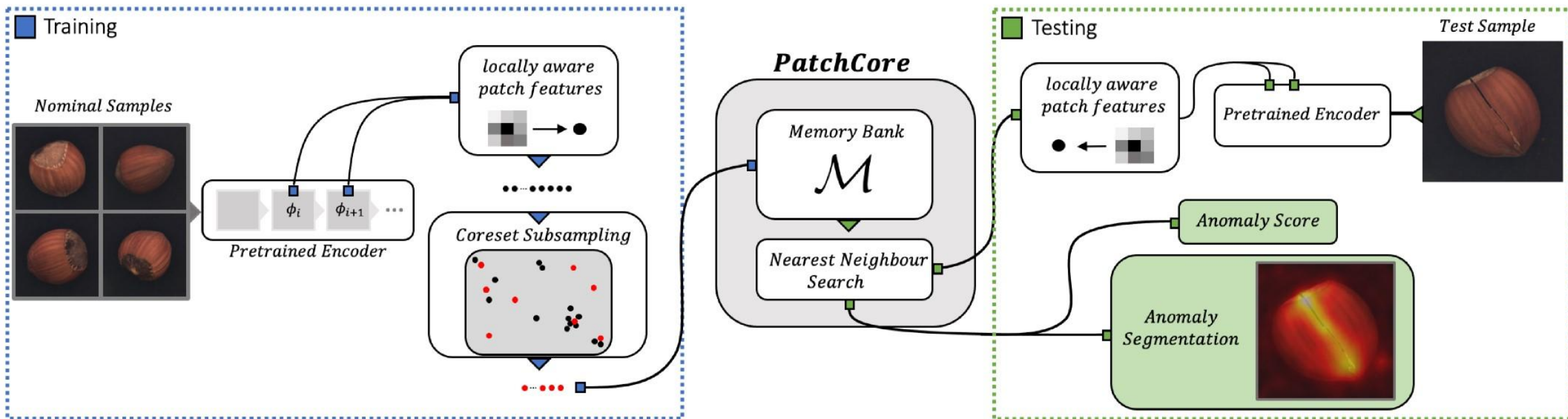
Detects defects by comparing normal samples to anomalous ones using patch-level representations.



Dataset : Train on normal Data Only

Trained only on normal samples to detect deviations at inference time.

Patchcore flow: extract features from image patches, reduce with coreset, and store in FAISS for anomaly score.



Inside PatchCore: The Core Algorithm in Three Steps

Feature extraction

- Extract feature maps from the image using a pretrained CNN, then flatten into patch vectors

Coreset selection

- Use K-Center Greedy / K-mean to select a diverse subset of training patches for memory efficiency.

Anomaly scoring

- Score test patches by computing nearest-neighbor distances to coreset patches. Flag anomaly if max distance exceeds threshold.

Mathematical Expression

$$f(x) \in \mathbb{R}^{H \times W \times D}$$

$$\mathcal{P}(x) = \{f(x)_{i,j} \in \mathbb{R}^D \mid 1 \leq i \leq H, 1 \leq j \leq W\}$$

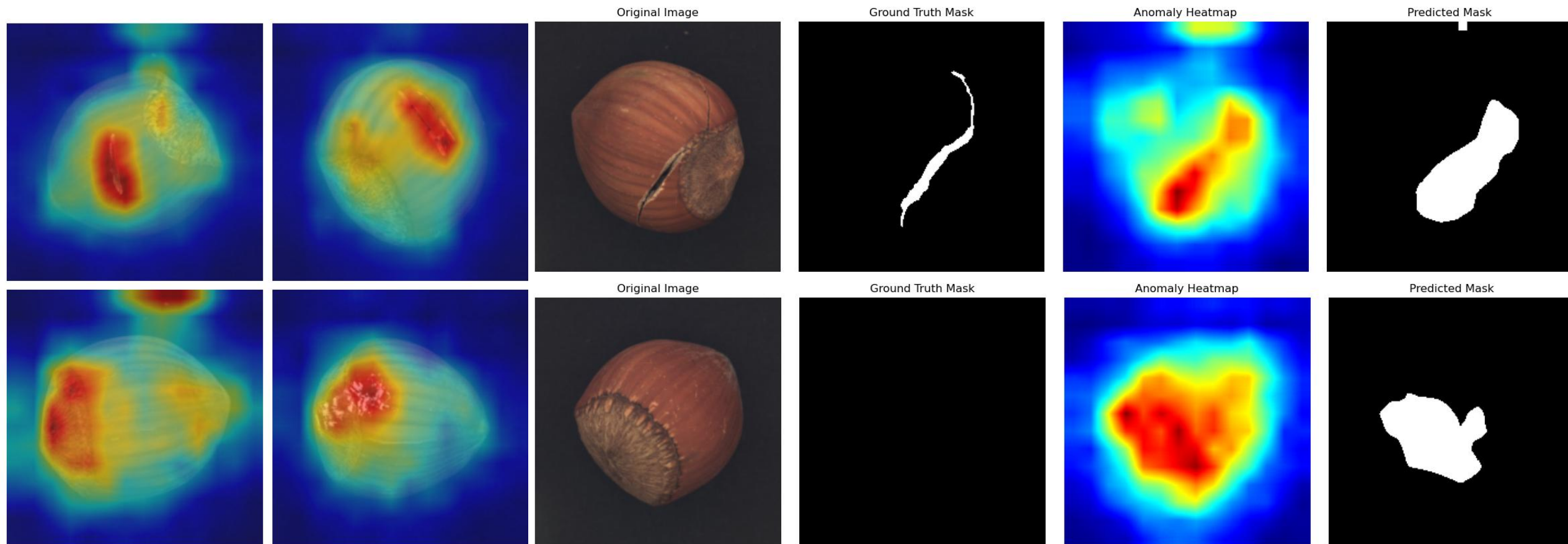
$$\mathcal{C} \subset \mathcal{P}_{\text{train}}$$

$$p = \arg \max_{p' \in \mathcal{P}} (\min_{q \in \mathcal{C}} \|p' - q\|_2)$$

$$d(p) = \min_{q \in \mathcal{C}} \|p - q\|_2$$

$$\max_{p \in \mathcal{P}(x)} d(p) > \text{threshold} \Rightarrow x \text{ is anomalous}$$

PatchCore on Hazelnut: able to highlight defected areas, but sensitive to rotation



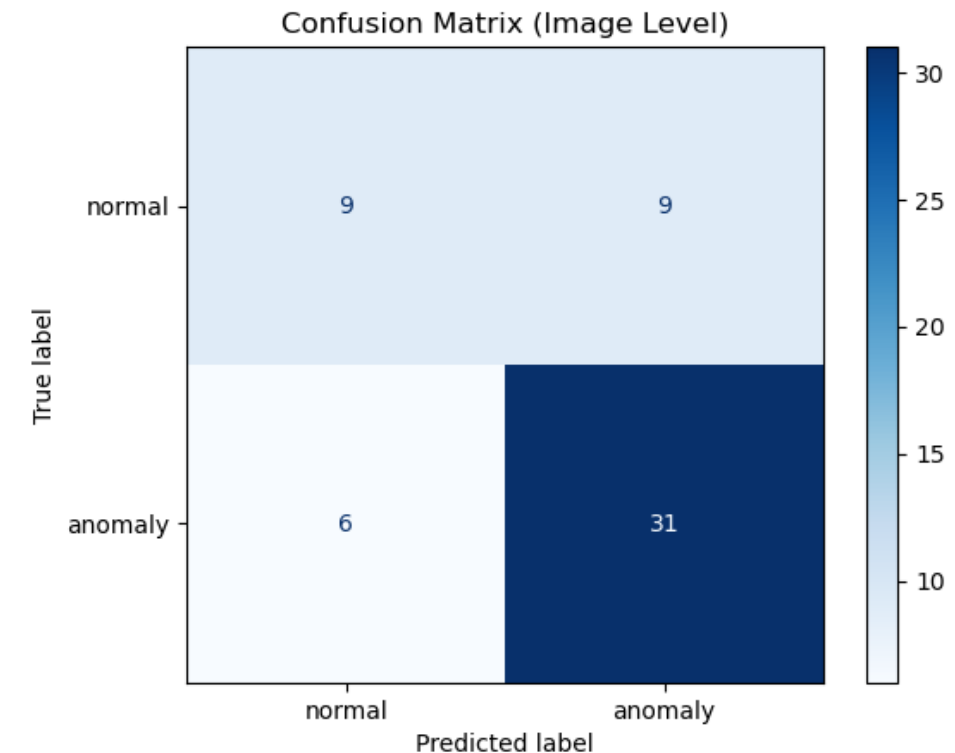
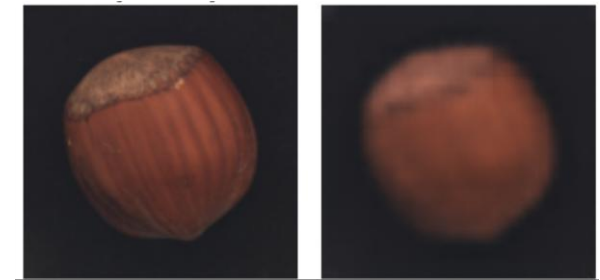
PatchCore on Hazelnut - results

Image level

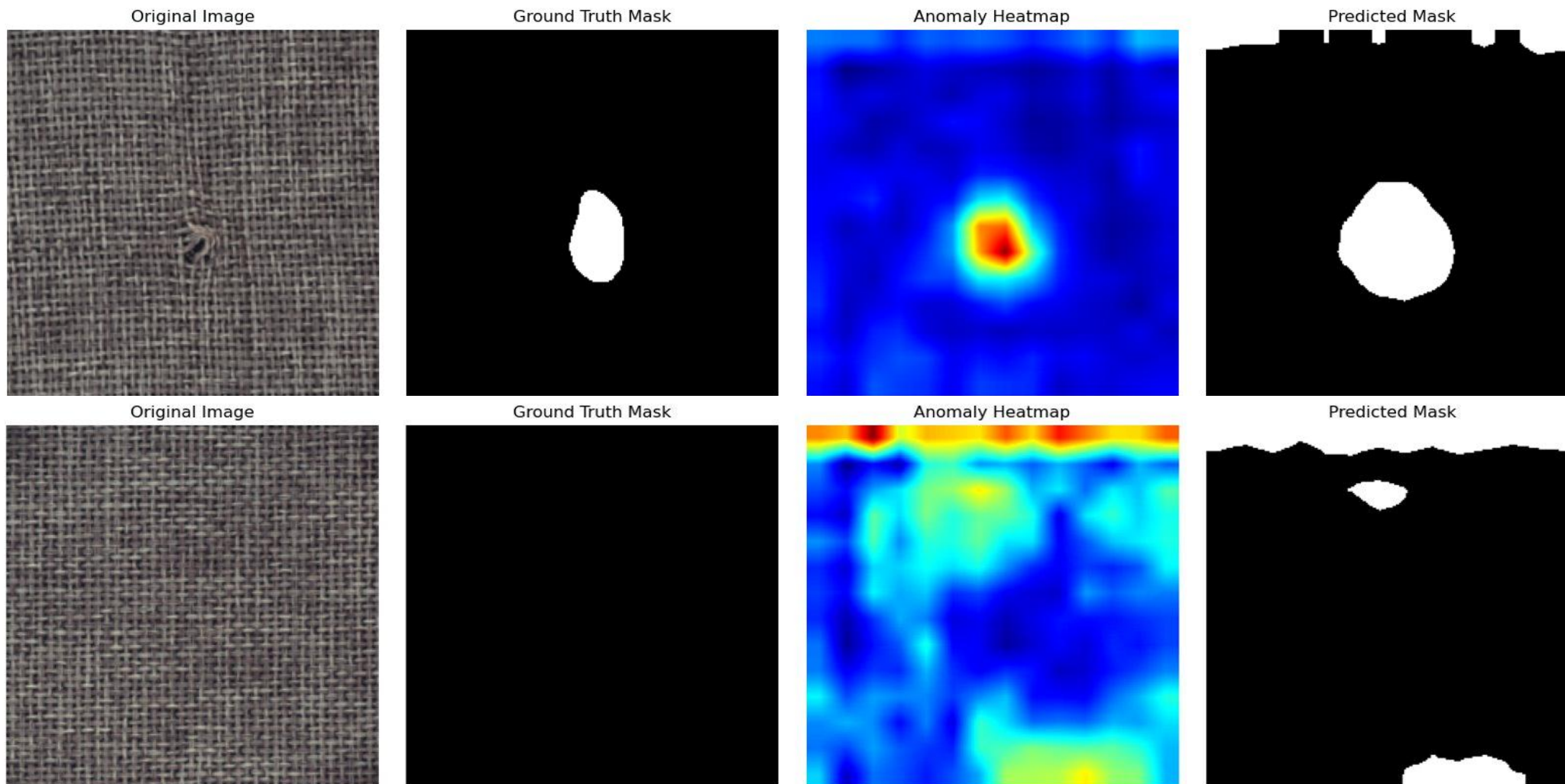
PR AUC	ROC AUC	F1 score	Precision	Recall
0.90	0.79	0.81	0.76	0.84

Pixel level

PR AUC	ROC AUC	F1 score	Precision	Recall
0.09	0.82	0.14	0.08	0.47



PatchCore on Carpet: able to highlight defected areas, but sensitive to rotation



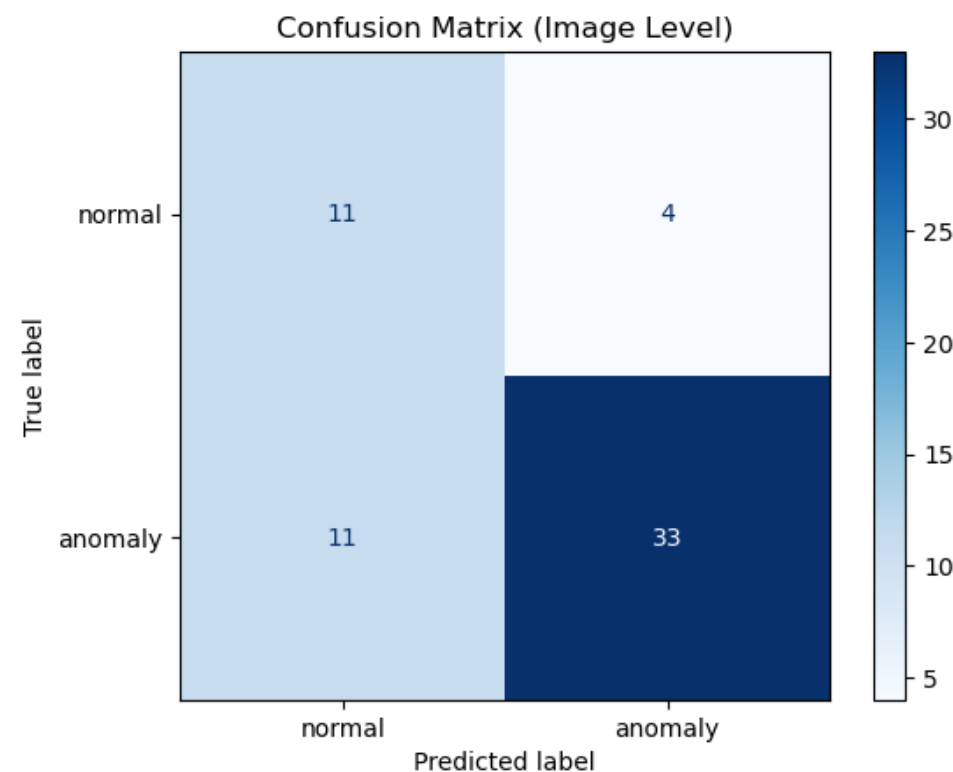
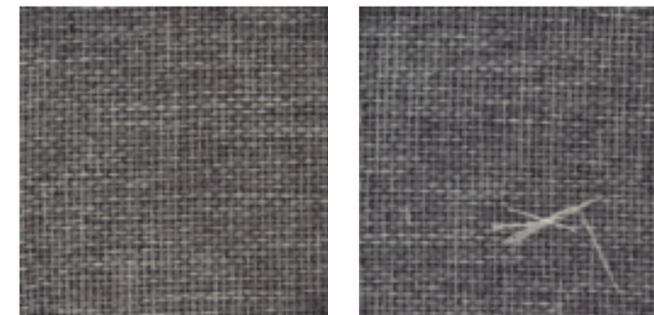
PatchCore on Carpet - results

Image level

PR AUC	ROC AUC	F1 score	Precision	Recall
0.90	0.77	0.81	0.89	0.75

Pixel level

PR AUC	ROC AUC	F1 score	Precision	Recall
0.03	0.65	0.07	0.04	0.42



PatchCore conclusion



Advantages

1. Only requires “normal” data (unsupervised learning).
2. Lightweight, fast
3. High accuracy
4. Provide defect maps at patch/ pixel level



Disadvantages

1. Sensitive to positional shifts
2. Depend on pretrained CNN
3. Have to finetune for every category

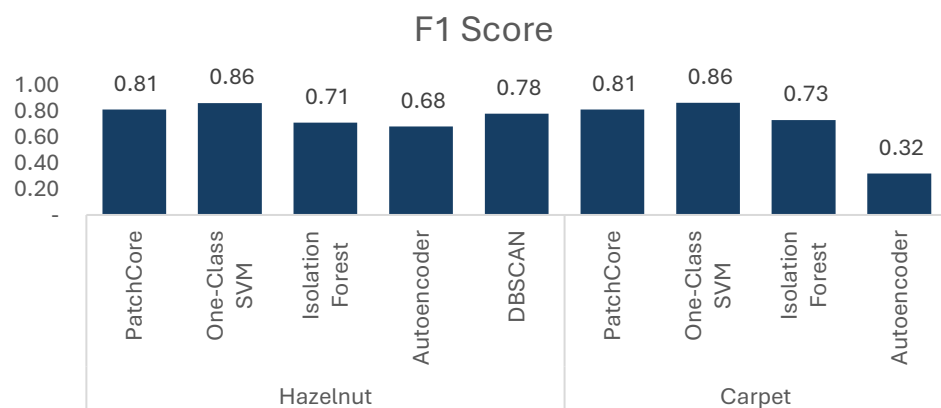
- **Conclusion: Despite being conceptually simple (frozen backbone + nearest neighbor), PatchCore is among the top performers on MVTec AD — a benchmark where many complex models fall short.**
 - Unlike deep models like CNN autoencoders, PatchCore requires **no training**. It uses **frozen pretrained CNNs** and still matches or outperforms many supervised and self-supervised approaches.
 - PatchCore generates detailed **pixel-level anomaly maps**, outperforming one-class classifiers (e.g., One-Class SVM, Deep SVDD) that only provide image-level labels --> allow operators to pinpoint defect locations immediately, crucial for repair/ replacement
 - PatchCore is **less robust** to rotations, scale, or lighting changes.

CONCLUSION

Conclusion

The recommended methods are as follows largely aligned with their image-level F1 performance:

- **Model Performance: PatchCore > One-Class SVM > Isolation Forest > Autoencoder > DBSCAN**
- **PatchCore** and **One-Class SVM** achieve the highest and most consistent F1 scores across both datasets.
- **Isolation Forest** follows with moderate performance and strong false-alarm control.
- **Autoencoder** performs well on Hazelnut but struggles on Carpet, yielding a lower average F1.
- **DBSCAN** cannot distinguish normals on its own embedding, resulting in unusable performance without domain-specific adaptation.



Dataset	Method	Accuracy	Precision	Recall	F1 Score	ROC AUC	PRAUC
Hazelnut	One-Class SVM	0.8	0.77	0.97	0.86	0.94	0.97
	Isolation Forest	0.71	0.98	0.56	0.71	0.90	0.95
	PatchCore	0.76	0.84	0.81	0.81	0.79	0.9
	Autoencoder	0.8	1	0.51	0.68	0.85	0.8
	DBSCAN	0.64	0.64	1	0.78	0.95	0.97
Carpet	One-Class SVM	0.76	0.77	0.99	0.86	0.6	0.85
	Isolation Forest	0.63	0.81	0.66	0.73	0.67	0.88
	PatchCore	0.89	0.75	0.81	0.81	0.77	0.9
	Autoencoder	0.52	0.31	0.32	0.32	0.5	0.42

Conclusion

Method	Characteristics	Strengths	Weaknesses
Autoencoder	learns a compressed representation	Tailored reconstruction error	High training cost, large data needed
One-Class SVM	finds a boundary around normal data	Kernel flexibility	Sensitive to kernel/threshold choices
Isolation Forest	isolates anomalies with fewer splits	Fast, interpretable anomaly scores	Over-conservative; misses subtle defects
DBSCAN	identifies noise as anomalies	Non-parametric density clustering	Requires well-separated embeddings
PatchCore	uses patch-level features & nearest neighbor search	Patch-level localization (anomaly maps)	Sensitive to rotation, scale, lighting

Domain-Specific Applications

- **Autoencoder:** Semiconductor wafer crack detection; medical image screening (CT/MRI)
- **One-Class SVM:** Network intrusion detection; fraud pattern identification in finance
- **Isolation Forest:** Real-time sensor anomaly alarms in manufacturing; IoT device health monitoring
- **DBSCAN:** Point-cloud defect clustering; clearly separated part-level quality control
- **PatchCore:** PCB solder-joint inspection with defect localization; robotic repair guidance in assembly lines



Limitations & Caution against Overgeneralization

Results show clear trends but are not exhaustive. More granular experiments, broader domains, and deeper optimization are needed before drawing definitive conclusions.

1. Data & Domain Constraints

- **Narrow Dataset Scope:** Only two MVTec-AD subsets (Hazelnut, Carpet)—other domains (e.g. medical, surveillance) untested.
- **Sample Size Limits:** Especially for Carpet, too few normal samples hinder deep-model learning and bias results.
- **Supervised Gap:** No minimal-label experiments (e.g. U-Net, defect-class classifiers) to gauge benefit from a few annotations.

2. Method & Resource Constraints

- **Hyperparameter & Preprocessing Depth:** Limited tuning and augmentations (rotation, lighting) leave room for robustness gains.
- **Excluded Advanced Approaches:** Resource-heavy methods (Stable Diffusion, DDPM synthesis, GAN-based models, self-supervised transformers) were not explored.
- **Methodological Coverage:** Only five unsupervised algorithms tested—many state-of-the-art detectors remain to be evaluated.

References

- AyZek_PhD. (2023, August 3). *Isolation Forest: Unveiling the Power of Unsupervised Anomaly Detection*. Medium. https://medium.com/@AyZek_PhD/isolation-forest-unveiling-the-power-of-unsupervised-anomaly-detection-b7c6f7130c42
- Sharma, P. (2020, May 20). *Isolation Forest*. Medium. <https://medium.com/@parulsharmmaa/isolation-forest-62fcb1de4cf5>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). *Isolation Forest*. In *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining* (pp. 413–422). IEEE. <https://doi.org/10.1109/ICDM.2008.17>

Appendix

Hazelnut Sample Images

hazelnut - good



hazelnut - good



hazelnut - good



hazelnut - crack



hazelnut - crack



hazelnut - crack



hazelnut - print



hazelnut - print



hazelnut - print



hazelnut - hole



hazelnut - hole



hazelnut - hole



hazelnut - cut



hazelnut - cut

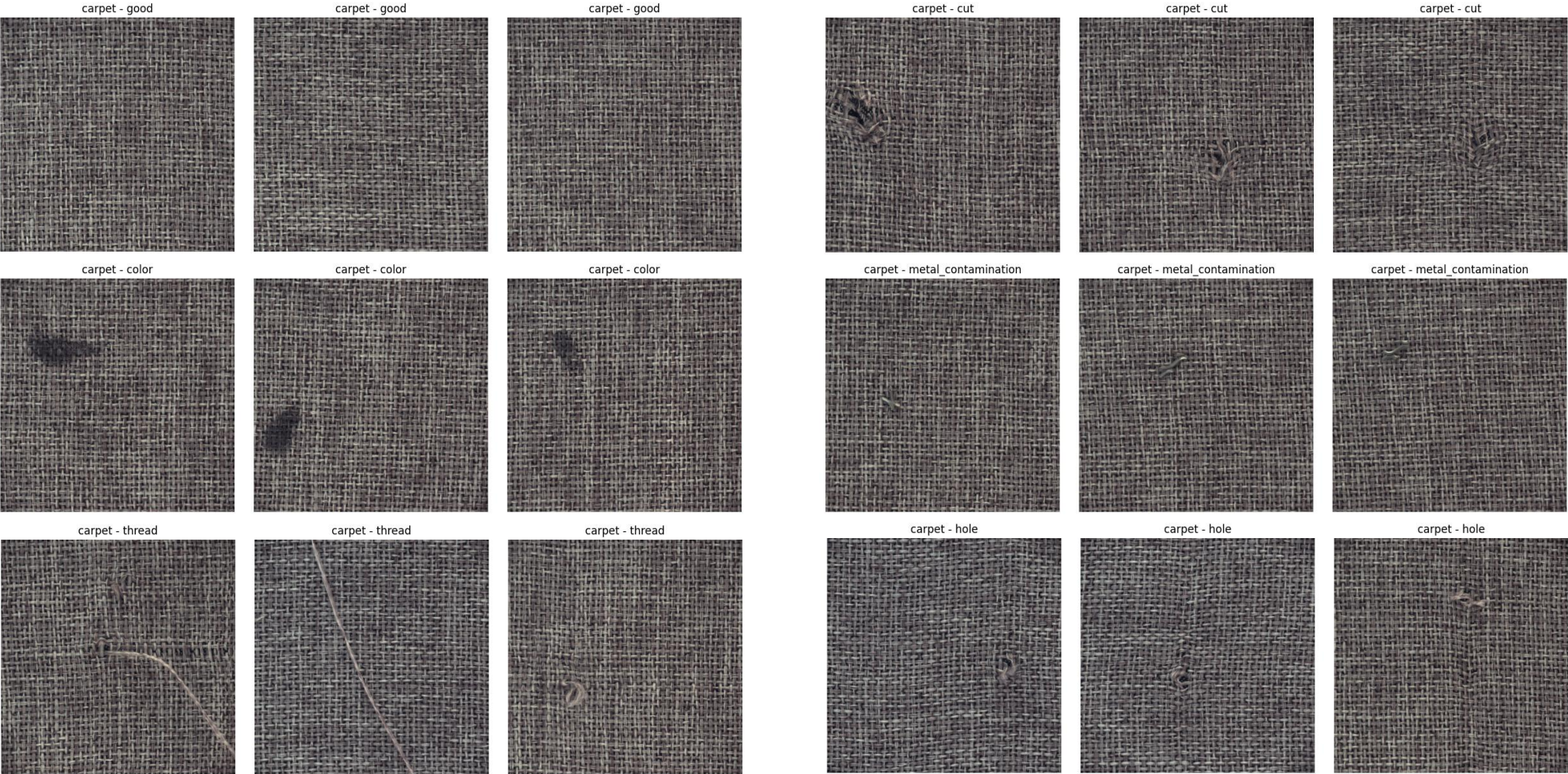


hazelnut - cut



Appendix

Carpet Sample Images



THANK YOU

